Chapter 2 ▪ Input and Output

In this chapter, we cover the Java Application Programming Interfaces (APIs) for input and output. You will learn how to access files and directories and how to read and write data in binary and text format. This chapter also shows you the object serialization mechanism that lets you store objects as easily as you can store text or numeric data. We finish the chapter with a discussion of regular expressions, even though they are not actually related to input and output. We couldn't find a better place to handle that topic, and apparently neither could the Java team—the regular expression API specification was attached to a specification request for "new I/O" features.

2.1. Input/Output Streams

In the Java API, an object from which we can read a sequence of bytes is called an *input stream.* An object to which we can write a sequence of bytes is called an *output stream.* These sources and destinations of byte sequences can be—and often are—files, but they can also be network connections and even blocks of memory. The abstract classes `InputStream` and `OutputStream` are the basis for a hierarchy of input/output (I/O) classes.



Note: These input/output streams are unrelated to the streams that you saw in the preceding chapter. For clarity, we will use the terms input stream, output stream, or input/output stream whenever we discuss streams that are used for input and output.

Byte-oriented input/output streams are inconvenient for processing information stored in Unicode (recall that Unicode uses multiple bytes per character). Therefore, a separate hierarchy provides classes, inheriting from the abstract `Reader` and `Writer` classes, for processing Unicode characters. These classes have read and write operations that are based on two-byte `char` values (that is, UTF-16 code units) rather than `byte` values.

2.1.1. Reading and Writing Bytes

The `InputStream` class has an abstract method:

```
abstract int read()
```

This method reads one byte and returns the byte that was read, or `-1` if it encounters the end of the input source. The designer of a concrete input stream class overrides this method to provide useful functionality. For example, in the `FileInputStream` class, this method reads one byte from a file. `System.in` is a predefined object of a subclass of `InputStream` that allows you to read information from "standard input," that is, the console or a redirected file.

The `InputStream` has a very useful method to read all bytes of a stream:

```
byte[] bytes = in.readAllBytes();
```

There are also methods to read a given number of bytes—see the API notes.

These methods call the abstract `read` method, so subclasses need to override only one method.

Similarly, the `OutputStream` class defines the abstract method

```
abstract void write(int b)
```

which writes one byte to an output location.

If you have an array of bytes, you can write them all at once:

```
byte[] values = . . .;
out.write(values);
```

The `transferTo` method transfers all bytes from an input stream to an output stream:

```
in.transferTo(out);
```

Both the `read` and `write` methods *block* until the byte is actually read or written. This means that if the input stream cannot immediately be accessed (usually because of a busy network connection), the current thread blocks. This gives other threads the chance to do useful work while the method is waiting for the input stream to become available again.

The `available` method lets you check the number of bytes that are currently available for reading. This means a fragment like the following is unlikely to block:

```
int bytesAvailable = in.available();
if (bytesAvailable > 0) {
    var data = new byte[bytesAvailable];
    in.read(data);
}
```

When you have finished reading or writing to an input/output stream, close it by calling the `close` method. This call frees up the operating system resources that are in limited supply. If an application opens too many input/output streams without closing them, system resources can become depleted. Closing an output stream also *flushes* the buffer used for the output stream: Any bytes that were temporarily placed in a buffer so that they could be delivered as a larger packet are sent off. In particular, if you do not close a file, the last packet of bytes might never be delivered. You can also manually flush the output with the `flush` method.

Even if an input/output stream class provides concrete methods to work with the raw `read` and `write` functions, application programmers rarely use them. The data that you are interested in probably contain numbers, strings, and objects, not raw bytes.

Instead of working with bytes, you can use one of many input/output classes that build upon the basic `InputStream` and `OutputStream` classes.

`java.io.InputStream` 1.0

- `abstract int read()`
  reads a byte of data and returns the byte read; returns `-1` at the end of the input stream.
- `int read(byte[] b)`
  reads into an array of bytes and returns the actual number of bytes read, or `-1` at the end of the input stream; this method reads at most `b.length` bytes.
- `int read(byte[] b, int off, int len)`
- `int readNBytes(byte[] b, int off, int len)` 9
  read up to `len` bytes, if available without blocking (`read`), or blocking until all values have been read (`readNBytes`). Values are placed into `b`, starting at `off`. Returns the actual number of bytes read.
- `byte[] readAllBytes()` 9
  yields an array of all bytes that can be read from this stream.
- `long transferTo(OutputStream out)` 9
  transfers all bytes from this input stream to the given output stream, returning the number of bytes transferred. Neither stream is closed.
- `long skip(long n)`
  attempts to skip `n` bytes in the input stream, returns the actual number of bytes skipped (which, for any reason, may be less than `n`).
- `long skipNBytes(long n)` 12
  skips `n` bytes in the input stream, returns the actual number of bytes skipped (which may be less than `n` if the end of the input stream was encountered).
- `int available()`
  returns the number of bytes available, without blocking (recall that blocking means that the current thread loses its turn).
- `void close()`
  closes the input stream.
- `void mark(int readlimit)`
  puts a marker at the current position in the input stream (not all streams support this feature). If more than `readlimit` bytes have been read from the input stream, the stream is allowed to forget the marker.
- `void reset()`
  returns to the last marker. Subsequent calls to `read` reread the bytes. If there is no current marker, the input stream is not reset.
- `boolean markSupported()`
  returns `true` if the input stream supports marking.
- `static InputStream nullInputStream()` 11
  returns an input stream with no bytes.

`java.io.OutputStream` 1.0

- `abstract void write(int n)`
  writes a byte of data.
- `void write(byte[] b)`
- `void write(byte[] b, int off, int len)`
  write all bytes, or `len` bytes starting at `off`, in the array `b`.
- `void close()`
  flushes and closes the output stream.

- `void flush()`
  flushes the output stream—that is, sends any buffered data to its destination.
- `static OutputStream nullOutputStream()` 11
  returns an output stream that discards all bytes.

2.1.2. The Complete Stream Zoo

Unlike C, which gets by just fine with a single type `FILE*`, Java has a whole zoo of more than 60 (!) classes and interfaces for input and output.

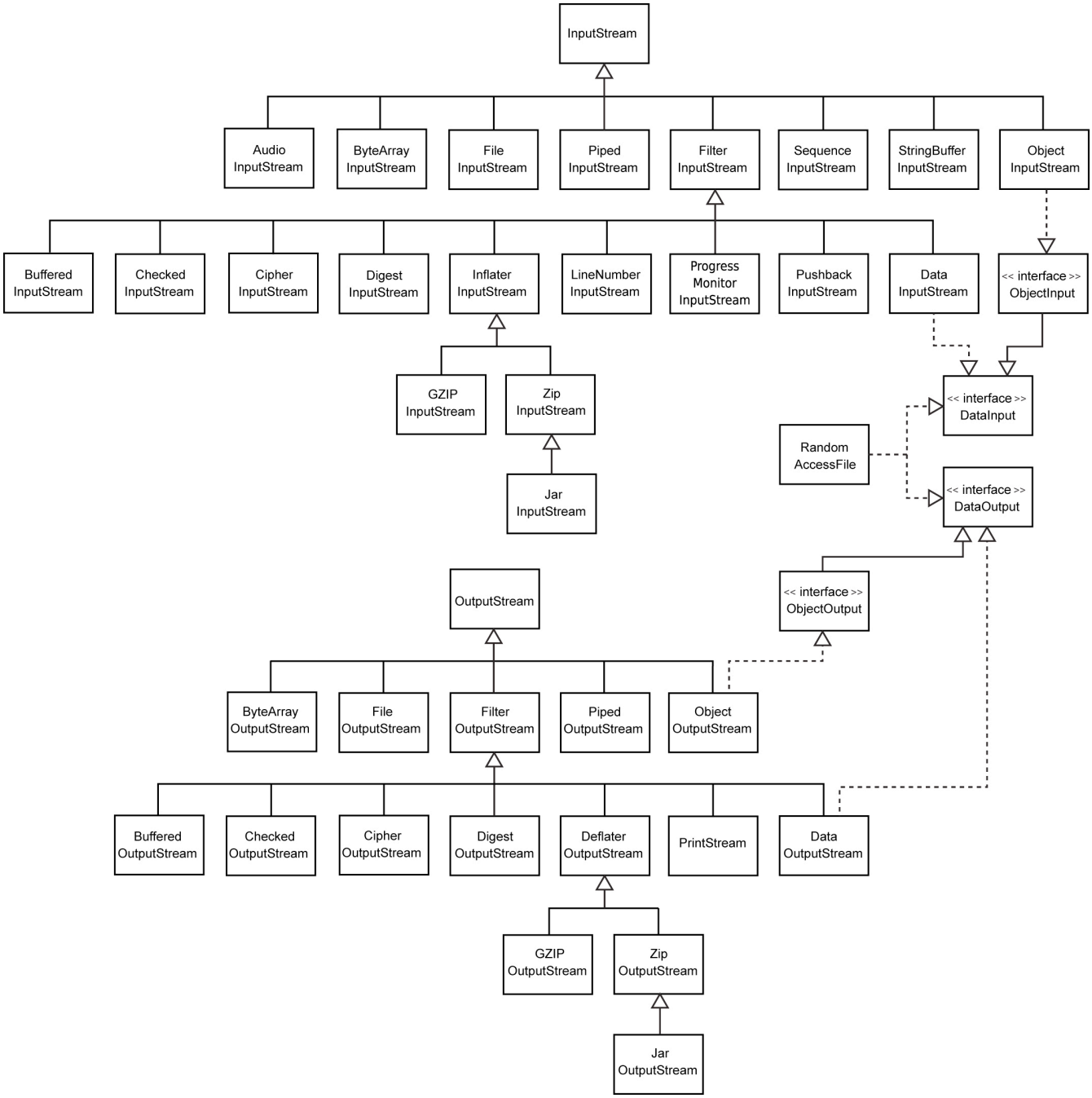


Figure 2.1: Input and output stream hierarchy

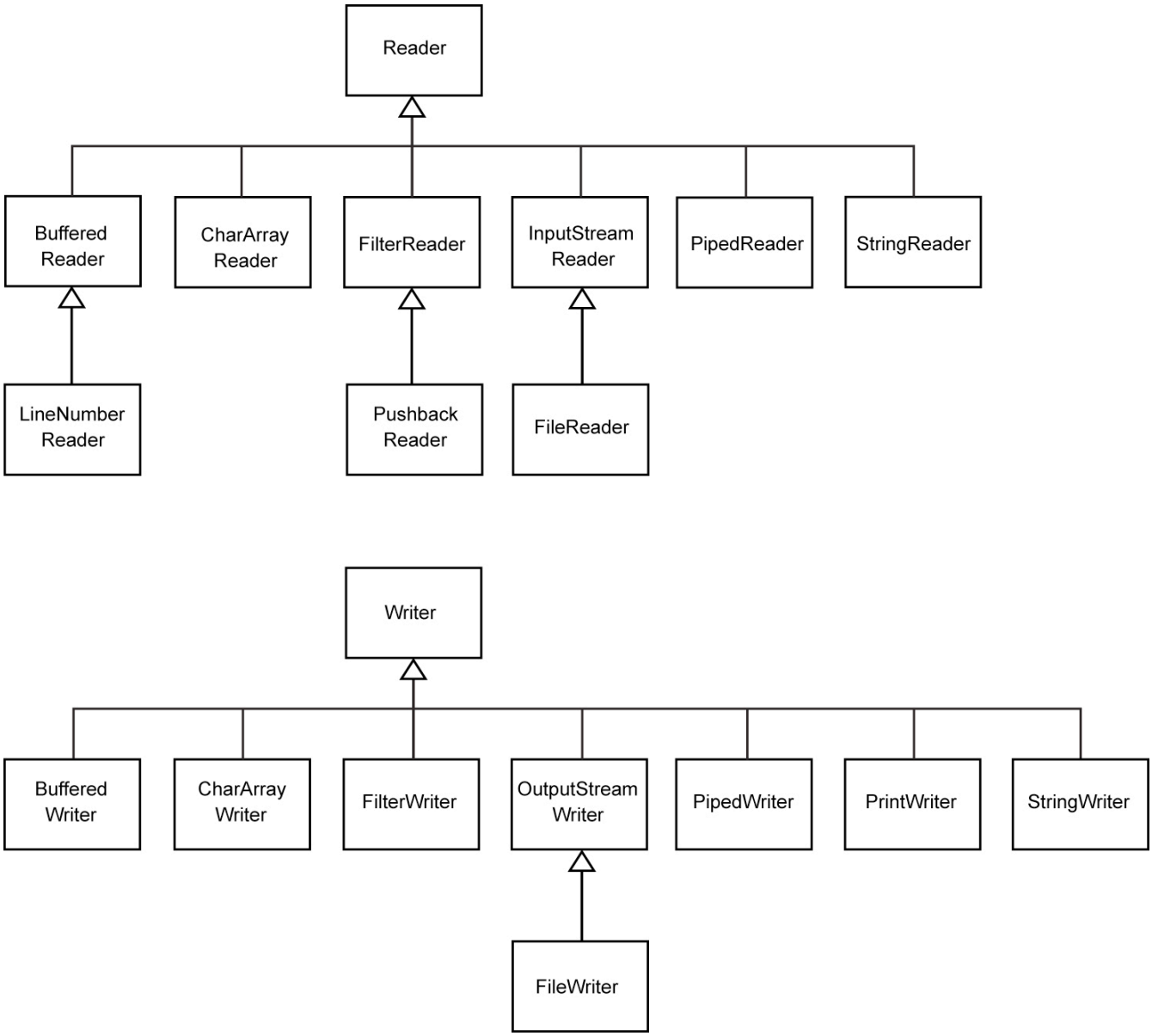


Figure 2.2: Reader and writer hierarchy

Let's divide the animals in the input/output zoo by how they are used. There are separate hierarchies for classes that process bytes and characters. As you saw, the `InputStream` and `OutputStream` classes let you read and write individual bytes and arrays of bytes. These classes form the basis of the hierarchy shown in **#ch01fig01**. To read and write strings and numbers, you need more capable subclasses. For example, `DataInputStream` and `DataOutputStream` let you read and write all the primitive Java types in binary format. Finally, there are input/output streams for specialized tasks; for example, the `ZipInputStream` and `ZipOutputStream` let you read and write files in the familiar ZIP compression format.

For Unicode text, on the other hand, you can use subclasses of the abstract classes `Reader` and `Writer` (see **#ch01fig02**). The basic methods of the `Reader` and `Writer` classes are similar to those of `InputStream` and `OutputStream`.

```
abstract int read()
abstract void write(int c)
```

The `read` method returns either a UTF-16 code unit (as an integer between `0` and `65535`) or `-1` when you have reached the end of the file. The `write` method is called with a Unicode code unit. (See **Chapter 3 of Volume I** for a discussion of Unicode code units.)

There are four additional interfaces: `Closeable`, `Flushable`, `Readable`, and `Appendable` (see #ch01fig03). The first two interfaces are very simple, with methods

```
void close() throws IOException
```

and

```
void flush()
```

respectively. The classes `InputStream`, `OutputStream`, `Reader`, and `Writer` all implement the `Closeable` interface.

Note: The `java.io.Closeable` interface extends the `java.lang.AutoCloseable` interface. Therefore, you can use the `try`-with-resources statement with any `Closeable`. Why have two interfaces? The `close` method of the `Closeable` interface only throws an `IOException`, whereas the `AutoCloseable.close` method may throw any exception.

`OutputStream` and `Writer` implement the `Flushable` interface.

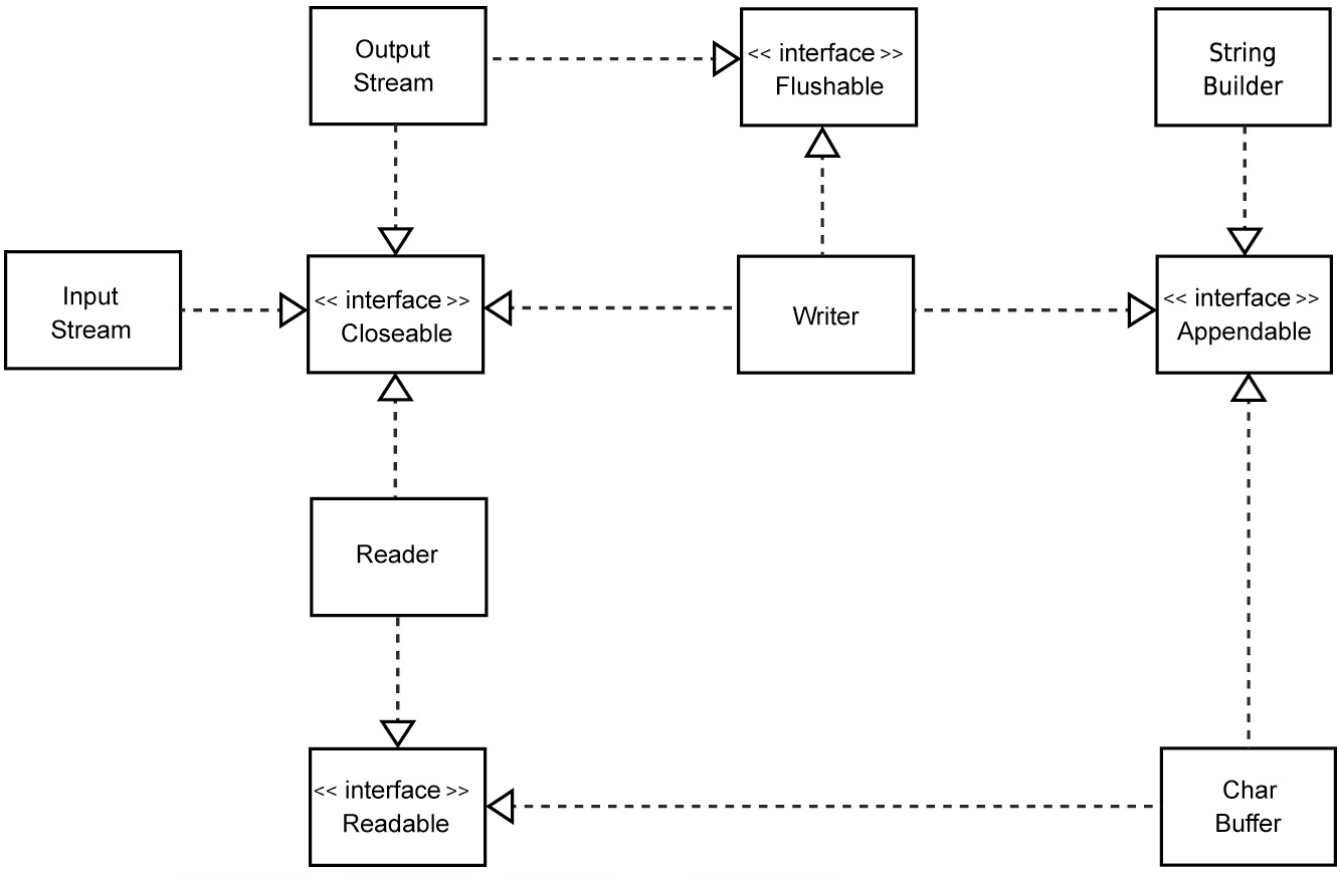


Figure 2.3: The `Closeable`, `Flushable`, `Readable`, and `Appendable` interfaces

The `Readable` interface has a single method

```
int read(CharBuffer cb)
```

The `CharBuffer` class has methods for sequential and random read/write access. It represents an in-memory buffer or a memory-mapped file. (See **Section 2.4.2** for details.)

The `Appendable` interface has two methods for appending single characters and character sequences:

```
Appendable append(char c)
Appendable append(CharSequence s)
```

The `CharSequence` interface describes basic properties of a sequence of `char` values. It is implemented by `String`, `CharBuffer`, `StringBuilder`, and `StringBuffer`.

Of the input/output stream classes, only `Writer` implements `Appendable`.

java.io.Closeable **5.0**

- `void close()`
  closes this `Closeable`. This method may throw an `IOException`.

java.io.Flushable **5.0**

- `void flush()`
  flushes this `Flushable`.

java.lang.Readable **5.0**

- `int read(CharBuffer cb)`
  attempts to read as many `char` values into `cb` as it can hold. Returns the number of values read, or `-1` if no further values are available from this `Readable`.

java.lang.Appendable **5.0**

- `Appendable append(char c)`
- `Appendable append(CharSequence cs)`
  append the given code unit, or all code units in the given sequence, to this `Appendable`; return `this`.

java.lang.CharSequence **1.4**

- `char charAt(int index)`
  returns the code unit at the given index.
- `int length()`
  returns the number of code units in this sequence.
- `CharSequence subSequence(int startIndex, int endIndex)`
  returns a `CharSequence` consisting of the code units stored from index `startIndex` to `endIndex - 1`.
- `String toString()`
  returns a string consisting of the code units of this sequence.

2.1.3. Combining Input/Output Stream Filters

`FileInputStream` and `FileOutputStream` give you input and output streams attached to a disk file. You need to pass the file name or full path name of the file to the constructor. For example,

```
var fin = new FileInputStream("employee.dat");
```

looks for a file named `employee.dat`.



Note: Relative path names start from the *working directory*. You can get this directory by a call to `System.getProperty("user.dir")`. It is set when the virtual machine starts and cannot be changed afterwards. If you start your program from the command line, the working directory is the current directory in the terminal. In an IDE, you can set the working directory in the run configuration.



Caution: Since the backslash character is the escape character in Java strings, be sure to use `\\` for Windows-style path names (for example, `C:\\Windows\\win.ini`). In Windows, you can also use a single forward slash (`C:/Windows/win.ini`) because most Windows file-handling system calls will interpret forward slashes as file separators. However, this is not recommended—the behavior of the Windows system functions is subject to change. Instead, for portable programs, use the file separator character for the platform on which your program runs. It is available as the constant string `java.io.File.separator`.

Like the abstract `InputStream` and `OutputStream` classes, these classes only support reading and writing at the byte level. That is, we can only read bytes and byte arrays from the object `fin`.

```
byte b = (byte) fin.read();
```

As you will see in the next section, if we just had a `DataInputStream`, we could read numeric types:

```
DataInputStream din = . . .;
double x = din.readDouble();
```

But just as the `FileInputStream` has no methods to read numeric types, the `DataInputStream` has no method to get data from a file.

Java uses a clever mechanism to separate two kinds of responsibilities. Some input streams (such as the `FileInputStream` and the input stream returned by the `openStream` method of the `URL` class) can retrieve bytes from files and other more exotic locations. Other input streams (such as the `DataInputStream`) can assemble bytes into more useful data types. The Java programmer has to combine the two. For example, to be able to read numbers from a file, first create a `FileInputStream` and then pass it to the constructor of a `DataInputStream`.

```
var fin = new FileInputStream("employee.dat");
var din = new DataInputStream(fin);
double x = din.readDouble();
```

If you look at **#ch01fig01** again, you can see the classes `FilterInputStream` and `FilterOutputStream`. The subclasses of these classes are used to add capabilities to input/output streams that process bytes.

You can add multiple capabilities by nesting the filters. For example, by default, input streams are not buffered. That is, every call to `read` asks the operating system to dole out yet another byte. It is more efficient to request

blocks of data instead and store them in a buffer. If you want buffering *and* the data input methods for a file, use the following rather monstrous sequence of constructors:

```
var din = new DataInputStream(
    new BufferedInputStream(
        new FileInputStream("employee.dat")));
```

Notice that we put the `DataInputStream` *last* in the chain of constructors because we want to use the `DataInputStream` methods, and we want *them* to use the buffered `read` method.

Sometimes you'll need to keep track of the intermediate input streams when chaining them together. For example, when reading input, you often need to peek at the next byte to see if it is the value that you expect. Java provides the `PushbackInputStream` for this purpose.

```
var pbin = new PushbackInputStream(
    new BufferedInputStream(
        new FileInputStream("employee.dat")));
```

Now you can speculatively read the next byte

```
int b = pbin.read();
```

and throw it back if it isn't what you wanted.

```
if (b != '<') pbin.unread(b);
```



Note: The `unread` method doesn't actually alter the source from which the stream reads its bytes. Unread bytes are stored in a buffer and are returned by the next `read` operation. By default, the buffer has length 1, but you can specify a longer buffer in the `PushbackInputStream` constructor.

Reading and unreading are the *only* methods that apply to a pushback input stream. If you want to look ahead and also read numbers, then you need both a pushback input stream and a data input stream reference.

```
var pbin = new PushbackInputStream(
    new BufferedInputStream(
        new FileInputStream("employee.dat")));
var din = new DataInputStream(pbin);
```

Of course, in the input/output libraries of other programming languages, niceties such as buffering and look-ahead are automatically taken care of—so it is a bit of a hassle to resort, in Java, to combining stream filters. However, the ability to mix and match filter classes to construct useful sequences of input/output streams does give you an immense amount of flexibility. For example, you can read numbers from a compressed ZIP file by using the following sequence of input streams (see **#ch01fig04**):

```
var zin = new ZipInputStream(new FileInputStream("employee.zip"));
var din = new DataInputStream(zin);
```

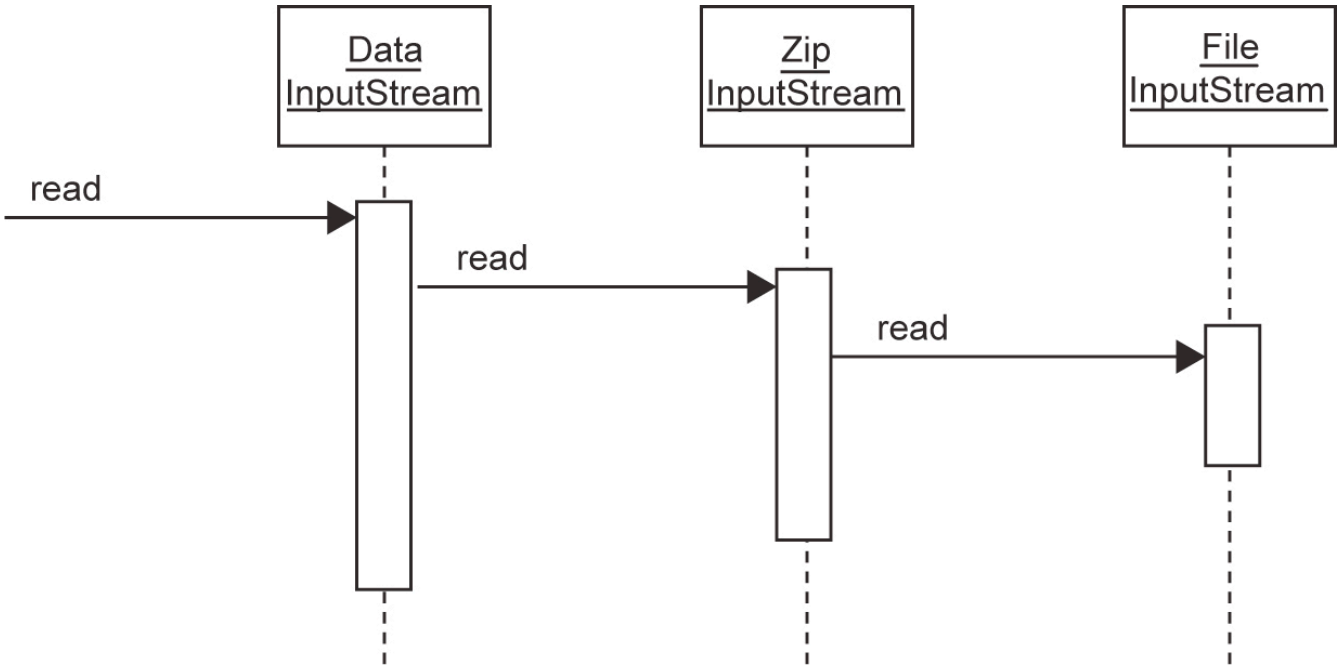


Figure 2.4: A sequence of filtered input streams

(See **Section 2.2.3** for more on Java's handling of ZIP files.)

`java.io.FileInputStream` 1.0

- `FileInputStream(String name)`
- `FileInputStream(File file)`

  create a new file input stream using the file whose path name is specified by the `name` string or the object of the legacy `File` class.

`java.io.FileOutputStream` 1.0

- `FileOutputStream(String name)`
- `FileOutputStream(String name, boolean append)`
- `FileOutputStream(File file)`
- `FileOutputStream(File file, boolean append)`

  create a new file output stream specified by the `name` string or the `file` object. (The `File` class is described at the end of this chapter.) If the `append` parameter is `true`, an existing file with the same name will not be deleted and data will be added at the end of the file. Otherwise, this method deletes any existing file with the same name.

`java.io.BufferedInputStream` 1.0

- `BufferedInputStream(InputStream in)`

  creates a buffered input stream. A buffered input stream reads bytes from a stream without causing a device access every time. When the buffer is empty, a new block of data is read into the buffer.

`java.io.BufferedOutputStream` 1.0

- `BufferedOutputStream(OutputStream out)`

  creates a buffered output stream. A buffered output stream collects bytes to be written without causing a device access every time. When the buffer fills up or when the stream is flushed, the data are written.

`java.io.PushbackInputStream` 1.0

- `PushbackInputStream(InputStream in)`
- `PushbackInputStream(InputStream in, int size)`
  construct an input stream with one-byte lookahead or a pushback buffer of specified size.
- `void unread(int b)`
  pushes back a byte, which is retrieved again by the next call to `read`.

2.1.4. Text Input and Output

When saving data, you have the choice between binary and text formats. For example, if the integer 1234 is saved in binary, it is written as the sequence of bytes `00 00 04 D2` (in hexadecimal notation). In text format, it is saved as the string `"1234"`. Although binary I/O is fast and efficient, it is not easily readable by humans. We first discuss text I/O and cover binary I/O in **Section 2.2**.

When saving text strings, you need to consider the *character encoding*. In the UTF-16 encoding that Java uses internally, the string `"José"` is encoded as `00 4A 00 6F 00 73 00 E9` (in hex). However, many programs expect that text files use a different encoding. In UTF-8, the encoding most commonly used on the Internet, the string would be written as `4A 6F 73 C3 A9`, without the zero bytes for the first three letters and with two bytes for the é character.

The `OutputStreamWriter` class turns an output stream of Unicode code units into a stream of bytes. Conversely, the `InputStreamReader` class turns an input stream that contains bytes (specifying characters in some character encoding) into a reader that emits Unicode code units.

Since Java 18, these classes use the UTF-8 encoding by default. Before Java 18, or if you need to support an archaic encoding, specify the encoding in the constructor, for example:

```
var in = new InputStreamReader(new FileInputStream("data.txt"), StandardCharsets.UTF_8);
```

See **Section 2.1.8** for more information on character encodings.

The `Reader` and `Writer` classes have only basic methods to read and write individual characters. As with streams, you use subclasses for processing strings and numbers.

2.1.5. How to Read Text Input

You can read a short text file into a string like this:

```
String content = Files.readString(path);
```

But if you want the file as a sequence of lines, call

```
List<String> lines = Files.readAllLines(path);
```

If the file is large, process the lines lazily as a `Stream<String>`:

```
try (Stream<String> lines = Files.lines(path)) {
    . . .
}
```



Note: If an `IOException` occurs as the stream fetches the lines, that exception is wrapped into an `UncheckedIOException` which is thrown out of the stream operation. This subterfuge is necessary because stream operations are not declared to throw any checked exceptions.

The easiest way to process arbitrary text is the `Scanner` class. This code processes all lines in a file:

```
try (var in = new Scanner(path)) {
    while (in.hasNextLine()) {
        String line = in.nextLine();
        // Process line
    }
}
```

You can also use a scanner to read *tokens*—strings that are separated by a delimiter. The default delimiter is whitespace. You can change the delimiter to any regular expression. For example, here is how to configure a scanner so that it uses any non-Unicode letters as delimiters.

```
Scanner in = . . .;
in.useDelimiter("\\PL+");
```

This scanner now accepts tokens consisting only of Unicode letters.

Calling the `next` method yields the next token:

```
while (in.hasNext()) {
    String word = in.next();
    . . .
}
```

Alternatively, you can obtain a stream of all tokens as

```
Stream<String> words = in.tokens();
```



Note: In legacy code, you may find code that reads lines by repeatedly calling the `readLine` method of a `BufferedReader`. Nowadays, there is no good reason to do that.

The `nextInt` and `nextDouble` methods yield the next integer or floating-point number in the input. Call `hasNextInt` or `hasNextDouble` first to make sure that the next token is a number.



Caution: The `next`, `nextInt`, and `nextDouble` methods only consume leading whitespace and one token. Trailing whitespace is left in the input. Keep this in mind if you also call to `nextLine`. If your input is:

```
29.95
Blackwell Toaster
```

and you call

```
double price = in.nextDouble();
double description = in.nextLine(); // ERROR
```

then `description` will be empty! The call to `nextLine` only reads the newline following the number.

Simply add another call to `nextLine` to consume the trailing whitespace.

2.1.6. How to Write Text Output

For text output, use a `PrintWriter`. That class has methods to print strings and numbers in text format. In order to print to a file, construct a `PrintWriter` from a file name:

```
var out = new PrintWriter("employee.txt");
```



Caution: Curiously, you cannot construct a `PrintWriter` from a `Path`.



Note: Before Java 18, be sure to specify the character encoding as a second argument:

```
var out = new PrintWriter("employee.txt", StandardCharsets.UTF_8);
```

Otherwise, the platform encoding is used, making your program's behavior dependent on the configuration of the host operating system..

To write to a `PrintWriter`, use the `print` and `println` methods. You can use these methods to print numbers (`int`, `short`, `long`, `float`, `double`), characters, `boolean` values, strings, and objects.

For example, consider this code:

```
String name = "Harry Hacker";
double salary = 75000;
out.print(name);
out.print(' ');
out.println(salary);
```

This writes the characters

```
Harry Hacker 75000.0
```

to the writer `out`. The characters are then converted to bytes and end up in the file `employee.txt`.

The `println` method adds the correct end-of-line character for the target system (`"\r\n"` on Windows, `"\n"` on UNIX) to the line. This is the string obtained by the call `System.getProperty("line.separator")`.

If the writer is set to *autoflush mode*, all characters in the buffer are sent to their destination whenever `println` is called. (Print writers are always buffered.) By default, autoflushing is *not* enabled. You can enable or disable autoflushing by using the `PrintWriter(Writer writer, boolean autoFlush)` constructor:

```
var out = new PrintWriter(
    new OutputStreamWriter(
        new FileOutputStream("employee.txt")),
    true); // autoflush
```

The `print` methods don't throw exceptions. You can call the `checkError` method to see if something went wrong with the output stream.



Note: Java veterans might wonder what happened to the `PrintStream` class and to `System.out`. In Java 1.0, the `PrintStream` class simply truncated all Unicode characters to ASCII characters by dropping the top byte. (At the time, Unicode was still a 16-bit encoding.) Clearly, that was not a clean or portable approach, and it was fixed with the introduction of readers and writers in Java 1.1. For compatibility with existing code, `System.in`, `System.out`, and `System.err` are still input/output streams, not readers and writers. But now the `PrintStream` class internally converts Unicode characters to the default character encoding in the same way the `PrintWriter` does. Objects of type `PrintStream` act exactly like print writers when you use the `print` and `println` methods, but unlike print writers they allow you to output raw bytes with the `write(int)` and `write(byte[])` methods.

`java.io.PrintWriter` 1.1

- `PrintWriter(OutputStream out)`
- `PrintWriter(OutputStream out, boolean autoFlush)`
- `PrintWriter(Writer writer)`
- `PrintWriter(Writer writer, boolean autoFlush)`
  create a new `PrintWriter` that writes to the given output stream or writer. If `autoFlush` is specified as `true`, `println` and `printf` flush the output.
- `PrintWriter(String filename, String encoding)`
- `PrintWriter(File file, String encoding)`
  create a new `PrintWriter` that writes to the given file, using the given character encoding.
- `void print(Object obj)`
  prints an object by printing the string resulting from `toString`.
- `void print(String s)`
  prints a string containing Unicode code units.
- `void println(String s)`
  prints a string followed by a line terminator. Flushes the output stream if it is in autoflush mode.
- `void print(char[] s)`
  prints all Unicode code units in the given array.
- `void print(char c)`
  prints a Unicode code unit.

- `void print(int i)`
- `void print(long l)`
- `void print(float f)`
- `void print(double d)`
- `void print(boolean b)`
  print the given value in text format.
- `void printf(String format, Object... args)`
  prints the given values as specified by the format string. See **Chapter 3 of Volume I** for the specification of the format string.
- `boolean checkError()`
  returns `true` if a formatting or output error occurred. Once the output stream has encountered an error, it is tainted and all calls to `checkError` return `true`.

2.1.7. Saving Objects in Text Format

In this section, we discuss an example program that stores an array of `Employee` records in a text file. Each record is stored in a separate line. Instance fields are separated from each other by delimiters. A vertical bar (`|`) is used as a delimiter. (A colon (`:`) is another popular choice. Part of the fun is that everyone uses a different delimiter.) Naturally, we punt on the issue of what might happen if a `|` actually occurs in one of the strings we save.

Here is a sample set of records:

```
Harry Hacker|35500.0|1989-10-01
Carl Cracker|75000.0|1987-12-15
Tony Tester|38000.0|1990-03-15
```

Writing records is simple. Since we write to a text file, we use the `PrintWriter` class. We simply write all fields, followed by either a `|` or, for the last field, a newline character. This work is done in the following method:

```
public static void writeEmployee(PrintWriter out, Employee e) {
    out.println(e.getName() + "|" + e.getSalary() + "|" + e.getHireDay());
}
```

To read records, we read in a line at a time and separate the fields. We use a scanner to read each line and then split the line into tokens with the `String.split` method.

```
public static Employee readEmployee(Scanner in) {
    String line = in.nextLine();
    String[] tokens = line.split("\\|");
    String name = tokens[0];
    double salary = Double.parseDouble(tokens[1]);
    LocalDate hireDate = LocalDate.parse(tokens[2]);
    int year = hireDate.getYear();
    int month = hireDate.getMonthValue();
    int day = hireDate.getDayOfMonth();
    return new Employee(name, salary, year, month, day);
}
```

The parameter of the `split` method is a regular expression describing the separator. We discuss regular expressions in more detail at the end of this chapter. As it happens, the vertical bar character has a special meaning in regular expressions, so it needs to be escaped with a `\` character. That character needs to be escaped by another `\`, yielding the `"\\|"` expression.

The complete program is in **Listing 2.1**. The static method

```
void writeData(Employee[] e, PrintWriter out)
```

first writes the length of the array, then writes each record. The static method

```
Employee[] readData(Scanner in)
```

first reads in the length of the array, then reads in each record. This turns out to be a bit tricky:

```
int n = in.nextInt();
in.nextLine(); // consume newline
var employees = new Employee[n];
for (int i = 0; i < n; i++) {
    employees[i] = readEmployee(in);
}
```

The call to `nextInt` reads the array length but not the trailing newline character. We must consume the newline so that the `readEmployee` method can get the next input line when it calls the `nextLine` method.

Listing 2.1 v2ch02/textFile/TextFileDemo.java

```
1  package v2ch02.textFile;
2
3  import com.horstmann.corejava.*;
4  import module java.base;
5
6  class TextFileDemo {
7      void main() throws Exception {
8          var staff = new Employee[3];
9
10         staff[0] = new Employee("Carl Cracker", 75000, 1987, 12, 15);
11         staff[1] = new Employee("Harry Hacker", 50000, 1989, 10, 1);
12         staff[2] = new Employee("Tony Tester", 40000, 1990, 3, 15);
13
14         // save all employee records to the file employee.dat
15         try (var out = new PrintWriter("employee.dat")) {
16             writeData(staff, out);
17         }
18
19         // retrieve all records into a new array
20         try (var in = new Scanner(new FileInputStream("employee.dat"))) {
21             Employee[] newStaff = readData(in);
22
23             // print the newly read employee records
24             for (Employee e : newStaff)
```

```
                IO.println(e);
            }
        }

        void writeData(Employee[] employees, PrintWriter out) throws IOException {
            // write number of employees
            out.println(employees.length);

            for (Employee e : employees)
                writeEmployee(out, e);
        }

        Employee[] readData(Scanner in) {
            // retrieve the array size
            int n = in.nextInt();
            in.nextLine(); // consume newline

            var employees = new Employee[n];
            for (int i = 0; i < n; i++) {
                employees[i] = readEmployee(in);
            }
            return employees;
        }

        void writeEmployee(PrintWriter out, Employee e) {
            out.println(e.getName() + "|" + e.getSalary() + "|" + e.getHireDay());
        }

        Employee readEmployee(Scanner in) {
            String line = in.nextLine();
            String[] tokens = line.split("\\|");
            String name = tokens[0];
            double salary = Double.parseDouble(tokens[1]);
            LocalDate hireDate = LocalDate.parse(tokens[2]);
            int year = hireDate.getYear();
            int month = hireDate.getMonthValue();
            int day = hireDate.getDayOfMonth();
            return new Employee(name, salary, year, month, day);
        }
}
```

2.1.8. Character Encodings

Each Unicode "code point" has a 21-bit integer number. There are different *character encodings*—methods for packaging those 21-bit numbers into bytes.

The most common encoding is UTF-8, which encodes each Unicode code point into a sequence of one to four bytes. **Table 2.1** shows in detail how to encode a 21-bit code point with bits $a_{20}a_{19}. \ . \ .a_1a_0$. UTF-8 has the advantage that the characters of the traditional ASCII character set, which contains all characters used in English, only take up one byte each.

Table 2.1: UTF-8 Encoding

Character Range	Encoding

Character Range	Encoding
Top 14 bits zero	$0a_6a_5a_4a_3a_2a_1a_0$
Top ten bits zero	$110a_{10}a_9a_8a_7a_6\ 10a_5a_4a_3a_2a_1a_0$
Top six bits zero	$1110a_{15}a_{14}a_{13}a_{12}\ 10a_{11}a_{10}a_9a_8a_7a_6\ 10a_5a_4a_3a_2a_1a_0$
The remainder	$11110a_{20}a_{19}a_{18}\ 10a_{17}a_{16}a_{15}a_{14}a_{13}a_{12}\ 10a_{11}a_{10}a_9a_8a_7a_6\ 10a_5a_4a_3a_2a_1a_0$

Table 2.2: UTF-16 Encoding

Character Range	Encoding
Top six bits zero	$a_{15}a_{14}a_{13}a_{12}a_{11}a_{10}a_9a_8\ a_7a_6a_5a_4a_3a_2a_1a_0$
The remainder	$110110b_{19}b_{18}\ b_{17}b_{16}a_{15}a_{14}a_{13}a_{12}a_{11}a_{10}\ 110111a_9a_8\ a_7a_6a_5a_4a_3a_2a_1a_0$ where $b_{19}b_{18}b_{17}b_{16} = a_{20}a_{19}a_{18}a_{17}a_{16} - 1$

Another common encoding is UTF-16, which encodes each Unicode code point into one or two 16-bit values (see Table 2.2). This is the encoding used in Java strings. Actually, there are two forms of UTF-16, called "big-endian" and "little-endian." Consider the 16-bit value `0x2122`. In the big-endian format, the more significant byte comes first: `0x21` followed by `0x22`. In the little-endian format, it is the other way around: `0x22 0x21`. To indicate which of the two is used, a file can start with the "byte order mark" or BOM, the 16-bit quantity `0xFEFF`. A reader can use this value to determine the byte order and then discard it.



Caution: Some programs add a BOM at the beginning of UTF-8 encoded files. Clearly, this is unnecessary since there are no byte ordering issues in UTF-8. But the Unicode standard allows it, and some programmers consider it a good idea since it leaves little doubt about the encoding. Any BOM is supposed to be ignored when reading a UTF-8 encoded file. Sadly, Java does not do that, and bug reports against this issue are closed as "will not fix." Your best bet is to strip out any leading `\uFEFF` that you find in your input.

In addition to the UTF encodings, there are partial encodings that cover a character range suitable for a given user population. For example, ISO 8859-1 is a one-byte code that includes accented characters used in Western European languages. Shift-JIS is a variable-length code for Japanese characters. A number of these encodings are still in use.

There is no reliable way to automatically detect the character encoding from a stream of bytes. Sometimes, the encoding is specified elsewhere. For example, when reading a web page, check the `Content-Type` header.

The `StandardCharsets` class has static variables of type `Charset` for the character encodings that every Java virtual machine must support:

```
StandardCharsets.UTF_8
StandardCharsets.UTF_16
StandardCharsets.UTF_16BE
StandardCharsets.UTF_16LE
StandardCharsets.UTF_32
StandardCharsets.UTF_32BE
```

```
StandardCharsets.UTF_32LE
StandardCharsets.ISO_8859_1
StandardCharsets.US_ASCII
```

To obtain the `Charset` for another encoding, use the static `forName` method:

```
Charset shiftJIS = Charset.forName("Shift-JIS");
```

The static method `Charset.availableCharsets` returns all available `Charset` instances, as a map from canonical names to `Charset` objects.

Use a `Charset` object whenever you need to specify a character encoding. For example, you can turn an array of bytes into a string as

```
var str = new String(bytes, StandardCharsets.ISO_8859_1);
```



Tip: Many methods allow you to specify a character encoding as a string instead of a `CharSet`. For the standard character encodings, use the `StandardCharsets` constants, so that any spelling errors are caught at compile time.



Caution: Up to Java 17, calling `Charset.forName` with the name `"default"` yielded `StandardCharsets.US_ASCII`! Nowadays, the call results in an `UnsupportedCharsetException`.

2.1.9. Default Character Encodings

If you do not specify a character encoding, and an operation needs to convert between bytes and strings, it will pick a default. For many methods in the `java.nio.file` package, that default is UTF-8.

But other methods use the *default charset* when no character encoding is specified. These include:

- The methods of the `java.io` reader and writer classes
- The `String(byte[])` constructor and the `getBytes` method of the `String` class
- `Scanner` and `Formatter` in the `java.util` package
- `URLEncoder` and `URLDecoder` in the `java.net` package

Prior to Java 18, the default charset was the *native encoding*, determined from the system executing the Java virtual machine. On Linux and Mac OS, that is UTF-8, unless it was specifically configured otherwise. However, on Windows, the native encoding is in most cases an archaic encoding such as Cp1252. (As of Java 17, the system property `native.encoding` yields the name of the native encoding.)

Prior to Java 18, the Oracle implementation of Java had a system property `file.encoding` for overriding the default charset. This was not an officially supported property, but it was nevertheless widely used.

As of Java 18, this property has become standardized. You can set the `file.encoding` property on the command line to these values:

- `COMPAT`: Sets the default charset to the native encoding, and the `file.encoding` system property to its name.
- `UTF-8`: Sets the default charset to UTF-8.

- Another character encoding such as `US-ASCII`: For some unspecified legacy charset names, it may set the default charset to the given one.



Caution: The `file.encoding` property must be set on the command line. Setting it via `System.setProperty` does not update the default charset.

The actually used default charset is returned by the static method `Charset.defaultCharset`.

Finally, the Windows console uses an even more archaic character encoding than the one used for file encoding. You may want to override that when redirecting output to a file. As of Java 18, you can do so by setting the system properties `stdout.encoding` and `stderr.encoding`.

The naming of these system properties is perhaps confusing. The encoding does not apply to `stdout` and `stderr` (which are byte streams) but to the `System.out` and `System.err` objects. Both are instances of `PrintStream`, whose `print` and `println` methods need an encoding that turns characters to bytes.

Java 25 adds the `stdin.encoding` property, specifying the encoding when reading from `System.in`. It is used by the `IO` and `Console` classes, but not by `Scanner`.



Tip: To quickly see your current settings for the various encodings, run:

```
java -XshowSettings:properties
```

Look for the properties whose names end in `.encoding`. Here are typical property values for Java 25:

Property	Windows	Linux/Mac OS
`native.encoding`	`Cp1252`	`UTF-8`
`file.encoding`	`UTF-8`	`UTF-8`
`stdin.encoding`	`cp437`	`UTF-8`
`stdout.encoding`	`cp437`	`UTF-8`
`stderr.encoding`	`cp437`	`UTF-8`

2.1.10. Reading Character Input

If you read a file with a structured format such as JSON or XML, you will use a parser that someone wrote who understands the fiddly details of that format. Such a parser typically reads a character at a time.

In the uncommon case that you need to implement such a parser, use a `BufferedReader` for efficiency. Keep calling its `read` method, which yields a `char` value or -1 at the end of input. The reader yields UTF-16 code units. If you are interested in code points, you need to handle the UTF-16 encoding:

```
int ch = reader.read();
if (ch != -1) {
    int codePoint;
    if (Character.isHighSurrogate((char) ch)) {
```

```
            int ch2 = reader.read();
            if (Character.isLowSurrogate((char) ch2))
                codePoint = Character.toCodePoint(ch, ch2);
            else
                throw new MalformedInputException(2);
        }
        else
            codePoint = ch;
    }
```

The `Character` class contains methods to tell whether a particular code point has a given property. For example,

```
Character.isLetter(codePoint)
```

returns `true` if `codePoint` is a letter in some language. Here are some other classification methods:

```
isUpperCase
isLowerCase
isDigit
isSpaceChar
isEmoji
```

These methods use the rules of the Unicode standard. Others refer to the rules of the Java language:

```
isJavaIdentifierStart
isJavaIdentifierPart
isWhitespace
```

After analyzing the code points, you often need to store them in strings, converting them back to UTF-16. The `appendCodePoint` method of the `StringBuilder` class turns a code point into one or two `char` values which are appended to the builder.

2.2. Reading and Writing Binary Data

Text format is convenient for testing and debugging because it is humanly readable, but it is not as efficient as transmitting data in binary format. In the following sections, you will learn how to perform input and output with binary data.

2.2.1. The `DataInput` and `DataOutput` Interfaces

The `DataOutput` interface defines the following methods for writing a number, a character, a `boolean` value, or a string in binary format:

```
writeChars          writeFloat
writeByte           writeDouble
writeInt            writeChar
writeShort          writeBoolean
writeLong           writeUTF
```

For example, `writeInt` always writes an integer as a 4-byte binary quantity regardless of the number of digits, and `writeDouble` always writes a `double` as an 8-byte binary quantity. The resulting output is not human-readable, but it will use the same space for each value of a given type and reading it back in will be faster than parsing text.



Note: There are two different methods of storing integers and floating-point numbers in memory, depending on the processor you are using. Suppose, for example, you are working with a 4-byte `int`, such as the decimal number 1234, or 4D2 in hexadecimal (1234 = 4 × 256 + 13 × 16 + 2). This value can be stored in such a way that the first of the four bytes in memory holds the most significant byte (MSB) of the value: `00 00 04 D2`. This is the so-called big-endian method. Or, we can start with the least significant byte (LSB) first: `D2 04 00 00`. This is called, naturally enough, the little-endian method. For example, the SPARC uses big-endian; Intel processors, little-endian. This can lead to problems. When a file is saved from a C or C++ file, the data are saved exactly as the processor stores them. That makes it challenging to move even the simplest data files from one platform to another. In Java, all values are written in the big-endian fashion, regardless of the processor. That makes Java data files platform-independent.

The `writeUTF` method writes string data using a modified version of the 8-bit Unicode Transformation Format. Instead of simply using the standard UTF-8 encoding, sequences of Unicode code units are first represented in UTF-16, and then the result is encoded using the UTF-8 rules. This modified encoding is different for characters with codes higher than `0xFFFF`. It is used for backward compatibility with virtual machines that were built when Unicode had not yet grown beyond 16 bits.

Since nobody else uses this modification of UTF-8, you should only use the `writeUTF` method to write strings intended for a Java virtual machine—for example, in a program that generates bytecodes. Use the `writeChars` method for other purposes.

To read the data back in, use the following methods defined in the `DataInput` interface:

```
readInt             readDouble
readShort           readChar
readLong            readBoolean
readFloat           readUTF
```

The `DataInputStream` class implements the `DataInput` interface. To read binary data from a file, combine a `DataInputStream` with a source of bytes such as a `FileInputStream`:

```
var in = new DataInputStream(new FileInputStream("employee.dat"));
```

Similarly, to write binary data, use the `DataOutputStream` class that implements the `DataOutput` interface:

```
var out = new DataOutputStream(new FileOutputStream("employee.dat"));
```

java.io.DataInput **1.0**

- `boolean readBoolean()`
- `byte readByte()`
- `char readChar()`
- `double readDouble()`
- `float readFloat()`
- `int readInt()`
- `long readLong()`
- `short readShort()`
  read in a value of the given type.
- `void readFully(byte[] b)`
  reads bytes into the array `b`, blocking until all bytes are read.
- `void readFully(byte[] b, int off, int len)`
  places up to `len` bytes into the array `b`, starting at `off`, blocking until all bytes are read.
- `String readUTF()`
  reads a string of characters in the "modified UTF-8" format.
- `int skipBytes(int n)`
  skips `n` bytes, blocking until all bytes are skipped.

java.io.DataOutput **1.0**

- `void writeBoolean(boolean b)`
- `void writeByte(int b)`
- `void writeChar(int c)`
- `void writeDouble(double d)`
- `void writeFloat(float f)`
- `void writeInt(int i)`
- `void writeLong(long l)`
- `void writeShort(int s)`
  write a value of the given type.
- `void writeChars(String s)`
  writes all characters in the string.
- `void writeUTF(String s)`
  writes a string of characters in the "modified UTF-8" format.

2.2.2. Random-Access Files

The `RandomAccessFile` class lets you read or write data anywhere in a file. Disk files are random-access, but input/output streams that communicate with a network socket are not. You can open a random-access file either for reading only or for both reading and writing; specify the option by using the string `"r"` (for read access) or `"rw"` (for read/write access) as the second argument in the constructor.

```
var in = new RandomAccessFile("employee.dat", "r");
var inOut = new RandomAccessFile("employee.dat", "rw");
```

There are also modes `"rwd"` and `"rws"` that flush updates of the data, or the data and metadata, to disk. This is slower but can prevent data loss when the program or computer crashes.

When you open an existing file as a `RandomAccessFile`, it does not get deleted.

A random-access file has a *file pointer* that indicates the position of the next byte to be read or written. The `seek` method can be used to set the file pointer to an arbitrary byte position within the file, specified as a `long` value.

The `getFilePointer` method returns the current position of the file pointer.

The `RandomAccessFile` class implements both the `DataInput` and `DataOutput` interfaces. To read and write from a random-access file, use methods such as `readInt`/`writeInt` and `readChar/writeChar` that we discussed in the preceding section.

Let's walk through an example program that stores employee records in a random-access file. Each record will have the same size. This makes it easy to read an arbitrary record. Suppose you want to position the file pointer to the third record. Simply set the file pointer to the appropriate byte position and start reading.

```
long n = 3;
in.seek((n - 1) * RECORD_SIZE);
var e = new Employee();
e.readData(in);
```

If you want to modify the record and save it back into the same location, remember to set the file pointer back to the beginning of the record:

```
in.seek((n - 1) * RECORD_SIZE);
e.writeData(out);
```

To determine the total number of bytes in a file, use the `length` method. The total number of records is the length divided by the size of each record.

```
long nbytes = in.length(); // length in bytes
int nrecords = (int) (nbytes / RECORD_SIZE);
```

Integers and floating-point values have a fixed size in binary format, but we have to work harder for strings. We provide two helper methods to write and read strings of a fixed size.

The `writeFixedString` writes the specified number of code units, starting at the beginning of the string. If there are too few code units, the method pads the string, using zero values.

```
public static void writeFixedString(String s, int size, DataOutput out)
        throws IOException {
    for (int i = 0; i < size; i++) {
        char ch = 0;
        if (i < s.length()) ch = s.charAt(i);
        out.writeChar(ch);
    }
}
```

The `readFixedString` method reads characters from the input stream until it has consumed `size` code units or until it encounters a character with a zero value. Then, it skips past the remaining zero values in the input field. For added efficiency, this method uses the `StringBuilder` class to read in a string.

```
public static String readFixedString(int size, DataInput in)
        throws IOException {
    var b = new StringBuilder(size);
    int i = 0;
    var done = false;
    while (!done && i < size) {
        char ch = in.readChar();
        i++;
        if (ch == 0) done = true;
        else b.append(ch);
    }
    in.skipBytes(2 * (size - i));
    return b.toString();
}
```

We placed the `writeFixedString` and `readFixedString` methods inside the `DataIO` helper class.

To write a fixed-size record, we simply write all fields in binary.

```
DataIO.writeFixedString(e.getName(), Employee.NAME_SIZE, out);
out.writeDouble(e.getSalary());
LocalDate hireDay = e.getHireDay();
out.writeInt(hireDay.getYear());
out.writeInt(hireDay.getMonthValue());
out.writeInt(hireDay.getDayOfMonth());
```

Reading the data back is just as simple.

```
String name = DataIO.readFixedString(Employee.NAME_SIZE, in);
double salary = in.readDouble();
int y = in.readInt();
int m = in.readInt();
int d = in.readInt();
```

Let us compute the size of each record. We will use 40 characters for the name strings. Therefore, each record will contain 100 bytes:

- 40 characters = 80 bytes for the name
- 1 `double` = 8 bytes for the salary
- 3 `int` = 12 bytes for the date

The program shown in **Listing 2.2** writes three records into a data file and then reads them from the file in reverse order. To do this efficiently requires random access—we need to get to the last record first.

Listing 2.2 v2ch02/randomAccess/RandomAccessDemo.java

```
package v2ch02.randomAccess;

import module java.base;
import com.horstmann.corejava.*;

public class RandomAccessDemo {
    final int NAME_SIZE = 40;
    final int RECORD_SIZE = 2 * NAME_SIZE + 8 + 4 + 4 + 4;

    void main() throws Exception {
        var staff = new Employee[3];

        staff[0] = new Employee("Carl Cracker", 75000, 1987, 12, 15);
        staff[1] = new Employee("Harry Hacker", 50000, 1989, 10, 1);
        staff[2] = new Employee("Tony Tester", 40000, 1990, 3, 15);

        try (var out = new DataOutputStream(new FileOutputStream("employee.dat"))) {
            // save all employee records to the file employee.dat
            for (Employee e : staff)
                writeData(out, e);
        }

        try (var in = new RandomAccessFile("employee.dat", "r")) {
            // retrieve all records into a new array

            // compute the array size
            int n = (int) (in.length() / RECORD_SIZE);
            var newStaff = new Employee[n];

            // read employees in reverse order
            for (int i = 0; i < n; i++) {
                in.seek((n - 1 - i) * RECORD_SIZE);
                newStaff[i] = readData(in);
            }

            // print the newly read employee records
            for (Employee e : newStaff)
                IO.println(e);
        }
    }

    /**
     * Writes employee data to a data output.
     * @param out the data output
     * @param e the employee
     */
    void writeData(DataOutput out, Employee e) throws IOException {
        DataIO.writeFixedString(e.getName(), NAME_SIZE, out);
        out.writeDouble(e.getSalary());

        LocalDate hireDay = e.getHireDay();
        out.writeInt(hireDay.getYear());
        out.writeInt(hireDay.getMonthValue());
        out.writeInt(hireDay.getDayOfMonth());
    }
```

```
56
57      /**
58       * Reads employee data from a data input.
59       * @param in the data input
60       * @return the employee
61       */
62      Employee readData(DataInput in) throws IOException {
63          String name = DataIO.readFixedString(NAME_SIZE, in);
64          double salary = in.readDouble();
65          int y = in.readInt();
66          int m = in.readInt();
67          int d = in.readInt();
68          return new Employee(name, salary, y, m, d);
69      }
70  }
```

`java.io.RandomAccessFile` 1.0

- `RandomAccessFile(String file, String mode)`
- `RandomAccessFile(File file, String mode)`
  open the given file for random access. The `mode` string is `"r"` for read-only mode, `"rw"` for read/write mode, `"rws"` for read/write mode with synchronous disk writes of data and metadata for every update, and `"rwd"` for read/write mode with synchronous disk writes of data only.
- `long getFilePointer()`
  returns the current location of the file pointer.
- `void seek(long pos)`
  sets the file pointer to `pos` bytes from the beginning of the file.
- `long length()`
  returns the length of the file in bytes.

2.2.3. ZIP Archives

ZIP archives store one or more files in a (usually) compressed format. Each ZIP archive has a header with information such as the name of each file and the compression method that was used. In Java, you can use a `ZipInputStream` to read a ZIP archive. You need to look at the individual *entries* in the archive. The `getNextEntry` method returns an object of type `ZipEntry` that describes the entry. Read from the stream until the end, which is actually the end of the current entry. Then call `closeEntry` to read the next entry. Do not close the `ZipInputStream` until you read the last entry. Here is a typical code sequence to read through a ZIP file:

```
var zin = new ZipInputStream(new FileInputStream(zipname));
boolean done = false;
while (!done) {
    ZipEntry entry = zin.getNextEntry();
    if (entry == null) done = true;
    else {
        read the contents of zin
        zin.closeEntry();
    }
}
zin.close();
```

To write a ZIP file, use a `ZipOutputStream`. For each entry that you want to place into the ZIP file, create a `ZipEntry` object. Pass the file name to the `ZipEntry` constructor; it sets the other parameters such as file date and decompression method. You can override these settings if you like. Then, call the `putNextEntry` method of the `ZipOutputStream` to begin writing a new file. Send the file data to the ZIP output stream. When done, call `closeEntry`. Repeat for all the files you want to store. Here is a code skeleton:

```
var fout = new FileOutputStream("test.zip");
var zout = new ZipOutputStream(fout);
for all files {
    var ze = new ZipEntry(filename);
    zout.putNextEntry(ze);
    send data to zout
    zout.closeEntry();
}
zout.close();
```



Note: JAR files (which were discussed in **Chapter 4 of Volume I**) are simply ZIP files with a special entry —the so-called manifest. Use the `JarInputStream` and `JarOutputStream` classes to read and write the manifest entry.

ZIP input streams are an example of the power of the stream abstraction. When you read data stored in compressed form, you don't need to worry that the data are being decompressed as they are being requested. Moreover, the source of the bytes in a ZIP stream need not be a file—the ZIP data can come from a network connection.



Note: **Section 2.3.9** shows how to access a ZIP archive without a special API, using the `FileSystem` class.

`java.util.zip.ZipInputStream` 1.1

- `ZipInputStream(InputStream in)`
  creates a `ZipInputStream` that allows you to inflate data from the given `InputStream`.
- `ZipEntry getNextEntry()`
  returns a `ZipEntry` object for the next entry, or `null` if there are no more entries.
- `void closeEntry()`
  closes the current open entry in the ZIP file. You can then read the next entry by using `getNextEntry()`.

`java.util.zip.ZipOutputStream` 1.1

- `ZipOutputStream(OutputStream out)`
  creates a `ZipOutputStream` that you can use to write compressed data to the specified `OutputStream`.
- `void putNextEntry(ZipEntry ze)`
  writes the information in the given `ZipEntry` to the output stream and positions the stream for the data. The data can then be written by calling the `write()` method.
- `void closeEntry()`
  closes the currently open entry in the ZIP file. Use the `putNextEntry` method to start the next entry.

- `void setLevel(int level)`
  sets the default compression level of subsequent `DEFLATED` entries to a value from `Deflater.NO_COMPRESSION` to `Deflater.BEST_COMPRESSION`. The default value is `Deflater.DEFAULT_COMPRESSION`. Throws an `IllegalArgumentException` if the level is not valid.
- `void setMethod(int method)`
  sets the default compression method for this `ZipOutputStream` for any entries that do not specify a method; can be either `DEFLATED` or `STORED`.

`java.util.zip.ZipEntry` 1.1

- `ZipEntry(String name)`
  constructs a ZIP entry with a given name.
- `long getCrc()`
  returns the CRC32 checksum value for this `ZipEntry`.
- `String getName()`
  returns the name of this entry.
- `long getSize()`
  returns the uncompressed size of this entry, or `-1` if the uncompressed size is not known.
- `boolean isDirectory()`
  returns `true` if this entry is a directory.
- `void setMethod(int method)`
  sets the compression method for the entry to `DEFLATED` or `STORED`.
- `void setSize(long size)`
  sets the size of this entry. Only required if the compression method is `STORED`.
- `void setCrc(long crc)`
  sets the CRC32 checksum of this entry. Use the `CRC32` class to compute this checksum. Only required if the compression method is `STORED`.

2.3. Working with Files

You have learned how to read and write data from a file. However, there is more to file management than reading and writing. The `Path` interface and `Files` class encapsulate the functionality required to work with the file system on the user's machine. For example, the `Files` class can be used to remove or rename a file, or to find out when a file was last modified. In other words, the input/output stream classes are concerned with the contents of files, whereas the classes that we discuss here are concerned with the storage of files on a disk.

The `Path` interface and `Files` class are much more convenient to use than the `File` class which dates back all the way to JDK 1.0. You should not use the `File` class except to interface with legacy code.

2.3.1. Paths

A `Path` is a sequence of directory names, optionally followed by a file name. The first component of a path may be a *root component* such as `/` or `C:\`. The permissible root components depend on the file system. A path that starts with a root component is *absolute*. Otherwise, it is *relative*. For example, here we construct an absolute and a relative path. For the absolute path, we assume a UNIX-like file system.

```
Path absolute = Path.of("/home", "harry");
Path relative = Path.of("myprog", "conf", "user.properties");
```

The static `Path.of` method receives one or more strings, which it joins with the path separator of the default file system (`/` for a UNIX-like file system, `\` for Windows). It then parses the result, throwing an `InvalidPathException` if the result is not a valid path in the given file system. The result is a `Path` object.

The `of` method can get a single string containing multiple components. For example, you can read a path from a configuration file like this:

```
String baseDir = props.getProperty("base.dir");
    // May be a string such as /opt/myprog or c:\Program Files\myprog
Path basePath = Path.of(baseDir); // OK that baseDir has separators
```



Note: A path does not have to correspond to a file that actually exists. It is merely an abstract sequence of names. As you will see in the next section, when you want to create a file, you first make a path and then call a method to create the corresponding file.

It is very common to combine or *resolve* paths. The call `p.resolve(q)` returns a path according to these rules:

- If `q` is absolute, then the result is `q`.
- if `q` does not have a root, then the result is obtained by joining `p` and `q`.
- Otherwise, the result depends on the rules of the file system.

For example, suppose your application needs to find its working directory relative to a given base directory that is read from a configuration file, as in the preceding example.

```
Path workRelative = Path.of("work");
Path workPath = basePath.resolve(workRelative);
```

There is a shortcut for the `resolve` method that takes a string instead of a path:

```
Path workPath = basePath.resolve("work");
```



Note: The `resolve` method does not simplify the result. For example,

```
Path.of("/usr/bin").resolve("../local")
```

is a path `/usr/bin/../local`.

There is a convenience method `resolveSibling` that resolves against a path's parent, yielding a sibling path. For example, if `workPath` is `/opt/myapp/work`, the call

```
Path tempPath = workPath.resolveSibling("temp");
```

creates `/opt/myapp/temp`.

The opposite of `resolve` is `relativize`. The call `p.relativize(r)` yields the path `q` which, when resolved with `p`, yields `r`. For example, relativizing `/home/harry` against `/home/fred/input.txt` yields

`../fred/input.txt`. Here, we assume that `..` denotes the parent directory in the file system.

The `normalize` method removes any redundant `.` and `..` components (or whatever the file system may deem redundant). For example, normalizing the path `/home/harry/../fred/./input.txt` yields `/home/fred/input.txt`.

The `toAbsolutePath` method yields the absolute path of a given path, starting at a root component, such as `/home/fred/input.txt` or `c:\Users\fred\input.txt`.

The `Path` interface has many useful methods for taking paths apart. This code sample shows some of the most useful ones:

```
Path p = Path.of("/home", "fred", "myprog.properties");
Path parent = p.getParent(); // the path /home/fred
Path file = p.getFileName(); // the path myprog.properties
Path root = p.getRoot(); // the path /
```



Caution: The `getFileName` method simply returns the last component of the given path. If the path describes a directory, that's a directory name, not a file name.

As you have already seen in Volume I, you can construct a `Scanner` from a `Path` object:

```
var in = new Scanner(Path.of("/home/fred/input.txt"));
```



Note: Occasionally, you may need to interoperate with legacy APIs that use the `File` class instead of the `Path` interface. The `Path` interface has a `toFile` method, and the `File` class has a `toPath` method.

java.nio.file.Path 7

- `static Path of(String first, String... more)` 11
  makes a path by joining the given strings.
- `Path resolve(Path other)`
- `Path resolve(String other)`
  If `other` is absolute, return `other`; otherwise, return the path obtained by joining `this` and `other`.
- `Path resolve(Path first, Path... more)` 22
- `Path resolve(String first, String... more)` 22
  resolve this path against `first`, then against the other paths.
- `Path resolveSibling(Path other)`
- `Path resolveSibling(String other)`
  If `other` is absolute, return `other`; otherwise, return the path obtained by joining the parent of `this` and `other`.
- `Path relativize(Path other)`
  returns the relative path that, when resolved with `this`, yields `other`.
- `Path normalize()`
  removes redundant path elements such as `.` and `..`
- `Path toAbsolutePath()`
  returns this path if it is absolute; otherwise, this path resolved against the working directory.

- `Path getParent()`
  returns the parent, or `null` if this path has no parent.
- `Path getFileName()`
  returns the last component of this path, or `null` if this path has no components.
- `Path getRoot()`
  returns the root component of this path, or `null` if this path has no root components.
- `toFile()`
  makes a `File` from this path.

`java.io.File` 1.0

- `Path toPath()` 7
  makes a `Path` from this file.

2.3.2. Reading and Writing Files

The `Files` class makes quick work of common file operations. For example, you can easily read the entire contents of a file as a byte array, string, list of lines, or stream of lines:

```
byte[] bytes = Files.readAllBytes(path);
String content = Files.readString(path);
List<String> lines = Files.readAllLines(path);
Stream<String> lineStream = Files.lines(path);
```

Conversely, for writing:

```
Files.write(path, bytes);
Files.writeString(path, content);
Files.write(path, lines);
```

To append to a given file, use

```
Files.write(path, content, StandardOpenOption.APPEND);
```

The call

```
long pos = Files.mismatch(path1, path2);
```

yields the position of the first byte where the file contents differ.

To get the MIME type of a file (such as `"text/html"` or `"image/png"`), call

```
String mimeType = Files.probeContentType(path);
```

The exact behavior depends on the Java implementation. Each implementation has a collection of `FileTypeDetector` instances that are tried in unspecified order. Each detector may examine the file exten-

sion, OS-specific metadata, or the first few bytes of the contents.

The following methods allow you to interact with an API that uses the classic input/output streams or readers/writers:

```
InputStream in = Files.newInputStream(path);
OutputStream out = Files.newOutputStream(path);
Reader in = Files.newBufferedReader(path);
Writer out = Files.newBufferedWriter(path);
```

These convenience methods save you from dealing with `FileInputStream`, `FileOutputStream`, `BufferedReader`, or `BufferedWriter`.

`java.nio.file.Files` 7

- `static byte[] readAllBytes(Path path)`
- `static String readString(Path path, Charset charset)`
- `static List<String> readAllLines(Path path, Charset charset)`
  read the contents of a file.
- `static Path write(Path path, byte[] contents, OpenOption... options)`
- `static Path write(Path path, String contents, Charset charset, OpenOption... options)`
- `static Path writeString(Path path, CharSequence contents, Charset cs, OpenOption... options)` 11
- `static Path write(Path path, Iterable<? extends CharSequence> contents, Charset cs, OpenOption options)`
  write the given contents to a file and return `path`.
- `static long mismatch(Path path, Path path2)` 12
  returns the position of the first byte where the file contents differ.
- `static String probeContentType(Path path)` 12
  returns the best guess of the installed file type detectors for the MIME type of the file contents, or `null` if the type could not be determined.
- `static InputStream newInputStream(Path path, OpenOption... options)`
- `static OutputStream newOutputStream(Path path, OpenOption... options)`
- `static BufferedReader newBufferedReader(Path path, Charset charset)`
- `static BufferedWriter newBufferedWriter(Path path, Charset charset, OpenOption... options)`
  open a file for reading or writing.

2.3.3. Creating Files and Directories

To create a new directory, call

```
Files.createDirectory(path);
```

All but the last component in the path must already exist. To create intermediate directories as well, use

```
Files.createDirectories(path);
```

You can create an empty file with

```
Files.createFile(path);
```

The call throws an exception if the file already exists. The check for existence and creation are atomic. If the file doesn't exist, it is created before anyone else has a chance to do the same.

There are convenience methods for creating a temporary file or directory in a given or system-specific location.

```
Path newPath = Files.createTempFile(dir, prefix, suffix);
Path newPath = Files.createTempFile(prefix, suffix);
Path newPath = Files.createTempDirectory(dir, prefix);
Path newPath = Files.createTempDirectory(prefix);
```

Here, `dir` is a `Path`, and `prefix`/`suffix` are strings which may be `null`. For example, the call `Files.createTempFile(null, ".txt")` might return a path such as `/tmp/1234405522364837194.txt`.

When you create a file or directory, you can specify attributes, such as owners or permissions. However, the details depend on the file system, and we won't cover them here.

`java.nio.file.Files` 7

- `static Path createFile(Path path, FileAttribute<?>... attrs)`
- `static Path createDirectory(Path path, FileAttribute<?>... attrs)`
- `static Path createDirectories(Path path, FileAttribute<?>... attrs)`
  create a file or directory. The `createDirectories` method creates any intermediate directories as well.
- `static Path createLink(Path link, Path existing)`
- `static Path createSymbolicLink(Path link, Path target, FileAttribute<?>... attrs)`
  creates a "hard" or symbolic link to the given target.
- `static Path createTempFile(String prefix, String suffix, FileAttribute<?>... attrs)`
- `static Path createTempFile(Path parentDir, String prefix, String suffix, FileAttribute<?>... attrs)`
- `static Path createTempDirectory(String prefix, FileAttribute<?>... attrs)`
- `static Path createTempDirectory(Path parentDir, String prefix, FileAttribute<?>... attrs)`
  create a temporary file or directory, in a location suitable for temporary files or in the given parent directory. Return the path to the created file or directory.

2.3.4. Copying, Moving, and Deleting Files

To copy a file from one location to another, simply call

```
Files.copy(fromPath, toPath);
```

To move the file instead, call

```
Files.move(fromPath, toPath);
```

The copy or move will fail if the target exists. If you want to overwrite an existing target, use the `REPLACE_EXISTING` option. If you want to copy all file attributes, use the `COPY_ATTRIBUTES` option. You can supply both like this:

```
Files.copy(fromPath, toPath, StandardCopyOption.REPLACE_EXISTING,
    StandardCopyOption.COPY_ATTRIBUTES);
```

You can specify that a move should be atomic. Then you are assured that either the move completed successfully, or the source continues to be present. Use the `ATOMIC_MOVE` option:

```
Files.move(fromPath, toPath, StandardCopyOption.ATOMIC_MOVE);
```

You can also copy an input stream to a `Path`, which just means saving the input stream to disk. Similarly, you can copy a `Path` to an output stream. Use the following calls:

```
Files.copy(inputStream, toPath);
Files.copy(fromPath, outputStream);
```

As with the other calls to `copy`, you can supply copy options as needed.

Finally, to delete a file, simply call

```
Files.delete(path);
```

This method throws an exception if the file doesn't exist, so instead you may want to use

```
boolean deleted = Files.deleteIfExists(path);
```

The deletion methods can also be used to remove an empty directory.

See **Table 2.4** for a summary of the options that are available for file operations.

Table 2.4: Standard Options for File Operations

Option	Description
`StandardOpenOption`; use with `newBufferedWriter`, `newInputStream`, `newOutputStream`, `write`.	
`READ`	Open for reading.
`WRITE`	Open for writing.
`APPEND`	If opened for writing, append to the end of the file.

Option	Description
`TRUNCATE_EXISTING`	If opened for writing, remove existing contents.
`CREATE_NEW`	Create a new file and fail if it exists.
`CREATE`	Atomically create a new file if it doesn't exist.
`DELETE_ON_CLOSE`	Make a “best effort” to delete the file when it is closed.
`SPARSE`	A hint to the file system that this file will be sparse.
`DSYNC or SYNC`	Requires that each update to the file data or data and metadata be written synchronously to the storage device.
`StandardCopyOption`; use with `copy`, `move`.	
`ATOMIC_MOVE`	Move the file atomically.
`COPY_ATTRIBUTES`	Copy the file attributes.
`REPLACE_EXISTING`	Replace the target if it exists.
`LinkOption`; use with all of the above methods and `exists`, `isDirectory`, `isRegularFile`.	
`NOFOLLOW_LINKS`	Do not follow symbolic links.
`FileVisitOption`; use with `find`, `walk`, `walkFileTree`	
`FOLLOW_LINKS`	Follow symbolic links.

`java.nio.file.Files` 7

- `static Path copy(Path from, Path to, CopyOption... options)`
- `static Path move(Path from, Path to, CopyOption... options)`
  copy or move `from` to the given target location and return `to`.
- `static long copy(InputStream from, Path to, CopyOption... options)`
- `static long copy(Path from, OutputStream to, CopyOption... options)`
  copy from an input stream to a file, or from a file to an output stream, returning the number of bytes copied.
- `static void delete(Path path)`
- `static boolean deleteIfExists(Path path)`
  delete the given file or empty directory. The first method throws an exception if the file or directory doesn't exist. The second method returns `false` in that case.

2.3.5. Getting File Information

The following static methods return a `boolean` value to check a property of a path:

- `exists`
- `isHidden`
- `isReadable`, `isWritable`, `isExecutable`
- `isRegularFile`, `isDirectory`, `isSymbolicLink`

The `size` method returns the number of bytes in a file.

```
long fileSize = Files.size(path);
```

The `getOwner` method returns the owner of the file, as an instance of `java.nio.file.attribute.UserPrincipal`.

All file systems report a set of basic attributes, encapsulated by the `BasicFileAttributes` interface, which partially overlaps with that information. The basic file attributes are

- The times at which the file was created, last accessed, and last modified, as instances of the class `java.nio.file.attribute.FileTime`
- Whether the file is a regular file, a directory, a symbolic link, or none of these
- The file size
- The file key—an object of some class, specific to the file system, that may or may not uniquely identify a file

To get these attributes, call

```
BasicFileAttributes attributes = Files.readAttributes(path, BasicFileAttributes.class);
```

If you know that the user's file system is POSIX-compliant, you can instead get an instance of `PosixFileAttributes`:

```
PosixFileAttributes attributes = Files.readAttributes(path, PosixFileAttributes.class);
```

Then you can find out the group owner and the owner, group, and world access permissions of the file. We won't dwell on the details since so much of this information is not portable across operating systems.

`java.nio.file.Files` 7

- `static boolean exists(Path path)`
- `static boolean isHidden(Path path)`
- `static boolean isReadable(Path path)`
- `static boolean isWritable(Path path)`
- `static boolean isExecutable(Path path)`
- `static boolean isRegularFile(Path path)`
- `static boolean isDirectory(Path path)`
- `static boolean isSymbolicLink(Path path)`
  check for the given property of the file given by the path.
- `static long size(Path path)`
  gets the size of the file in bytes.

- `A readAttributes(Path path, Class<A> type, LinkOption... options)`
  reads the file attributes of type `A`.

java.nio.file.attribute.BasicFileAttributes 7

- `FileTime creationTime()`
- `FileTime lastAccessTime()`
- `FileTime lastModifiedTime()`
- `boolean isRegularFile()`
- `boolean isDirectory()`
- `boolean isSymbolicLink()`
- `boolean isOther()`
- `long size()`
- `Object fileKey()`
  get the requested attribute.

`java.nio.file.attribute.FileTime` 7

- `Instant toInstant()` 8
  yields the `Instant` representing the same time as this `FileTime`.

2.3.6. Visiting Directory Entries

The static `Files.list` method returns a `Stream<Path>` that reads the entries of a directory. The directory is read lazily, making it possible to efficiently process directories with huge numbers of entries.

Since reading a directory involves a system resource that needs to be closed, you should use a `try` block:

```
try (Stream<Path> entries = Files.list(pathToDirectory)) {
    . . .
}
```

The `list` method does not enter subdirectories. To process all descendants of a directory, use the `Files.walk` method instead.

```
try (Stream<Path> entries = Files.walk(pathToRoot)) {
    // Contains all descendants, visited in depth-first order
}
```

Here is a sample traversal of the unzipped `src.zip` tree:

```
java
java/nio
java/nio/DirectCharBufferU.java
java/nio/ByteBufferAsShortBufferRL.java
java/nio/MappedByteBuffer.java
. . .
java/nio/ByteBufferAsDoubleBufferB.java
java/nio/charset
```

```
java/nio/charset/CoderMalfunctionError.java
java/nio/charset/CharsetDecoder.java
java/nio/charset/UnsupportedCharsetException.java
java/nio/charset/spi
java/nio/charset/spi/CharsetProvider.java
java/nio/charset/StandardCharsets.java
java/nio/charset/Charset.java
. . .
java/nio/charset/CoderResult.java
java/nio/HeapFloatBufferR.java
. . .
```

As you can see, whenever the traversal yields a directory, it is entered before continuing with its siblings.

You can limit the depth of the tree that you want to visit by calling `Files.walk(pathToRoot, depth)`. Both `walk` methods have a varargs parameter of type `FileVisitOption...`, but there is only one option you can supply: `FOLLOW_LINKS` to follow symbolic links.



Note: If you filter the paths returned by `walk` and your filter criterion involves the file attributes stored with a directory, such as size, creation time, or type (file, directory, symbolic link), then use the `find` method instead of `walk`. Call that method with a predicate function that accepts a path and a `BasicFileAttributes` object. The only advantage is efficiency. Since the directory is being read anyway, the attributes are readily available.

This code fragment uses the `Files.walk` method to copy one directory to another:

```
Files.walk(source).forEach(p -> {
    try {
        Path q = target.resolve(source.relativize(p));
        if (Files.isDirectory(p))
            Files.createDirectory(q);
        else
            Files.copy(p, q);
    }
    catch (IOException e) {
        throw new UncheckedIOException(e);
    }
});
```

Unfortunately, you cannot easily use the `Files.walk` method to delete a tree of directories since you need to delete the children before deleting the parent. Section 2.3.8 shows you how to overcome that problem.



Caution: The `Files.walk` method throws an exception if any of the subdirectories are not readable. If you only want to visit readable directories, use the `walkFileTree` method described in the next section.

2.3.7. Glob Patterns

Sometimes, it is useful to filter directory traversals by patterns. The *glob* syntax can be more useful for this than regular expressions. For example, the glob expression `**/*.java` denotes Java source files in any nested subdirectory.

Table 2.5 shows all glob patterns.

Table 2.5: Glob Patterns

Pattern	Description	Example
`*`	Matches zero or more characters of a path component.	`*.java` matches all Java files in the current directory.
`**`	Matches zero or more characters, crossing directory boundaries.	`**.java` matches all Java files in any subdirectory.
`?`	Matches one character.	`????.java` matches all four-character (not counting the extension) Java files.
`[...]`	Matches a set of characters. You can use hyphens `[0-9]` and negation `[!0-9]`.	`Test[0-9A-F].java` matches `Testx.java`, where x is one hexadecimal digit.
`{...}`	Matches alternatives, separated by commas.	`*.{java,class}` matches all Java and class files.
`\`	Escapes any of the above as well as `\`.	`*\**` matches all files with a * in their name.



Caution: If you use the glob syntax on Windows, you have to escape backslashes *twice*—once for the glob syntax, and once for the Java string syntax: `Files.newDirectoryStream(dir, "C:\\\\")`.

To filter by such a pattern, construct a `PathMatcher` and use its `matches` method.

```
PathMatcher javaSource = FileSystems.getDefault().getPathMatcher("glob:**/*.java");
List<Path> sourcePaths = Files.walk(root).filter(javaSource::matches).toList();
```

`java.nio.file.FileSystem` 7

- `PathMatcher getPathMatcher(String syntaxAndPattern)`
  Gets a matcher for the given syntax and pattern. The `syntaxAndPattern` string starts with a syntax name and a colon, then the pattern for that syntax. Every file system supports `glob` and `regex` syntax.

`java.nio.file.FileSystems` 7

- `static FileSystem getDefault()`
  gets the default file system.

`java.nio.file.PathMatcher` 7

- `boolean matches(Path path)`
  returns `true` if the given path matches.

2.3.8. Directory Streams

As you saw in **Section 2.3.6**, the `Files.walk` method produces a `Stream<Path>` that traverses the descendants of a directory. Sometimes, you need more fine-grained control over the traversal process. In that case, use the `Files.newDirectoryStream` object instead. It yields a `DirectoryStream`. Note that this is not a subinterface of `java.util.stream.Stream` but an interface that is specialized for directory traversal. It is a subinterface of `Iterable` so that you can use directory stream in an enhanced `for` loop. Here is the usage pattern:

```
try (DirectoryStream<Path> entries = Files.newDirectoryStream(dir)) {
    for (Path entry : entries)
        Process entries
}
```

The `try`-with-resources block ensures that the directory stream is properly closed.

There is no specific order in which the directory entries are visited.

You can filter the files with a glob pattern:

```
try (DirectoryStream<Path> entries = Files.newDirectoryStream(dir, "*.java"))
```

If you want to visit all descendants of a directory, call the `walkFileTree` method instead and supply an object of type `FileVisitor`. That object gets notified

- When a file is encountered: `FileVisitResult visitFile(T path, BasicFileAttributes attrs)`
- Before a directory is processed: `FileVisitResult preVisitDirectory(T dir, IOException e)`
- After a directory is processed: `FileVisitResult postVisitDirectory(T dir, IOException e)`
- When an error occurred trying to visit a file or directory, such as trying to open a directory without the necessary permissions: `FileVisitResult visitFileFailed(T path, IOException e)`

In each case, you can specify whether you want to

- Continue visiting the next file: `FileVisitResult.CONTINUE`
- Continue the walk, but without visiting the entries in this directory: `FileVisitResult.SKIP_SUBTREE`
- Continue the walk, but without visiting the siblings of this file: `FileVisitResult.SKIP_SIBLINGS`
- Terminate the walk: `FileVisitResult.TERMINATE`

If any of the methods throws an exception, the walk is also terminated, and that exception is thrown from the `walkFileTree` method.



Note: The `FileVisitor` interface is a generic type, but it isn't likely that you'll ever want something other than a `FileVisitor<Path>`. The `walkFileTree` method is willing to accept a `FileVisitor<? super Path>`, but `Path` does not have an abundance of supertypes.

A convenience class `SimpleFileVisitor` implements the `FileVisitor` interface with trivial methods that continue the visit or terminate it in case of an exception. Extend it and override the methods as needed.

For example, here is how to print out all subdirectories of a given directory:

```
Files.walkFileTree(Path.of("/"), new SimpleFileVisitor<Path>() {
        public FileVisitResult preVisitDirectory(Path path, BasicFileAttributes attrs)
                throws IOException {
            IO.println(path);
            return FileVisitResult.CONTINUE;
        }

        public FileVisitResult postVisitDirectory(Path dir, IOException e) {
            return FileVisitResult.CONTINUE;
        }

        public FileVisitResult visitFileFailed(Path path, IOException e)
                throws IOException {
            if (e != null) throw e;
            return FileVisitResult.SKIP_SUBTREE;
        }
    });
```

Note that we need to override `postVisitDirectory` and `visitFileFailed`. Otherwise, the visit would fail as soon as it encounters a directory that it's not allowed to open or a file it's not allowed to access.

Also note that the attributes of the path are passed as an argument to the `preVisitDirectory` and `visitFile` methods. The visitor already had to make an OS call to get the attributes, since it needs to distinguish between files and directories. This way, you don't need to make another call.

The other methods of the `FileVisitor` interface are useful if you need to do some work when entering or leaving a directory. For example, when you delete a directory tree, you need to remove the current directory after you have removed all of its files. Here is the complete code for deleting a directory tree:

```
// Delete the directory tree starting at root
Files.walkFileTree(root, new SimpleFileVisitor<Path>() {
        public FileVisitResult visitFile(Path file, BasicFileAttributes attrs)
                throws IOException {
            Files.delete(file);
            return FileVisitResult.CONTINUE;
        }

        public FileVisitResult postVisitDirectory(Path dir, IOException e)
                throws IOException {
            if (e != null) throw e;
            Files.delete(dir);
            return FileVisitResult.CONTINUE;
        }
    });
```

`java.nio.file.Files` 7

- `static DirectoryStream<Path> newDirectoryStream(Path path)`
- `static DirectoryStream<Path> newDirectoryStream(Path path, String glob)`

  get an iterator over the files and directories in a given directory. The second method only accepts those entries matching the given glob pattern.

- `static Path walkFileTree(Path start, FileVisitor<? super Path> visitor)`
  walks all descendants of the given path, applying the visitor to all descendants.

`java.nio.file.SimpleFileVisitor<T>` 7

- `static FileVisitResult visitFile(T path, BasicFileAttributes attrs)`
  is called when a file or directory is visited; returns one of `CONTINUE`, `SKIP_SUBTREE`, `SKIP_SIBLINGS`, or `TERMINATE`. The default implementation does nothing and continues.
- `static FileVisitResult preVisitDirectory(T dir, BasicFileAttributes attrs)`
- `static FileVisitResult postVisitDirectory(T dir, IOException exc)`
  are called before and after visiting a directory. The default implementations do nothing and continue. However, if `exc` is non-`null`, it is thrown, causing the visit to terminate with that exception.
- `static FileVisitResult visitFileFailed(T path, IOException exc)`
  is called if an exception was thrown in an attempt to get information about the given file. The default implementation rethrows the exception, which causes the visit to terminate with that exception. Override the method if you want to continue.

2.3.9. ZIP File Systems

The `Paths` class looks up paths in the default file system—the files on the user's local disk. You can have other file systems. One of the more useful ones is a *ZIP file system*. If `zipname` is the name of a ZIP file, then the call

```
FileSystem fs = FileSystems.newFileSystem(zippath);
```

establishes a file system that contains all files in the ZIP archive. It's an easy matter to copy a file out of that archive if you know its name:

```
Files.copy(fs.getPath(sourceName), targetPath);
```

Here, `fs.getPath` is the analog of `Path.of` for an arbitrary file system.

To list all files in a ZIP archive, walk the file tree:

```
Files.walk(fs.getPath("/")).forEach(IO::println);
```

To create a new ZIP file, call:

```
FileSystem fs = FileSystems.newFileSystem(zippath, Map.of("create", "true"));
```

Then copy files into the file system. When you are done, call

```
fs.close();
```

in order to write the changes to disk.

The ZIP file system is nicer than the API described in **Section 2.2.3** which required special classes just to deal with ZIP archives.

`java.nio.file.FileSystems` 7

- `static FileSystem newFileSystem(Path path)` 13
- `static FileSystem newFileSystem(Path path, Map<String, ?> env)` 13
  return the file system created by the first file system provider that accepts the given path. By default, there is a provider for ZIP file systems that accepts files whose names end in `.zip` or `.jar`. Use the `env` map for configuration parameters that depend on the file system provider.
- `void close()`
  Closes this file system and permanently writes any changes.

`java.nio.file.FileSystem` 7

- `static Path getPath(String first, String... more)`
  makes a path by joining the given strings.

2.4. Memory-Mapped Files

Most operating systems can take advantage of a virtual memory implementation to "map" a file, or a region of a file, into memory. Then the file can be accessed as if it were an in-memory array, which is much faster than the traditional file operations.

2.4.1. Memory-Mapped File Performance

At the end of this section, you can find a program that computes the CRC32 checksum of a file using traditional file input and a memory-mapped file. On one machine, we got the timing data shown in **Table 2.6** when computing the checksum of the 51MB file `src.zip` in the `lib` directory of the JDK.

Table 2.6: Timing Data for File Operations

Method	Time
Plain input stream	15.1 seconds
Buffered input stream	0.8 seconds
Random access file	11.9 seconds
Memory-mapped file	0.2 seconds

As you can see, on this particular machine, memory mapping is a bit faster than using buffered sequential input and dramatically faster than using a `RandomAccessFile`.

Of course, the exact values will differ greatly from one machine to another, but it is obvious that the performance gain, compared to random access, can be substantial.

Memory mapping uses a slightly less intuitive API than random access files. Here is what you do.

First, get a *channel* for the file. A channel is an abstraction for a disk file that lets you access operating system features such as memory mapping, file locking, and fast data transfers between files.

```
FileChannel channel = FileChannel.open(path, options);
```

Then, get a `ByteBuffer` from the channel by calling the `map` method of the `FileChannel` class. Specify the area of the file that you want to map and a *mapping mode*. Three modes are supported:

- `FileChannel.MapMode.READ_ONLY`: The resulting buffer is read-only. Any attempt to write to the buffer results in a `ReadOnlyBufferException`.
- `FileChannel.MapMode.READ_WRITE`: The resulting buffer is writable, and the changes will be written back to the file at some time. Note that other programs that have mapped the same file might not see those changes immediately. The exact behavior of simultaneous file mapping by multiple programs depends on the operating system.
- `FileChannel.MapMode.PRIVATE`: The resulting buffer is writable, but any changes are private to this buffer and not propagated to the file.

For example,

```
MappedByteBuffer buffer = channel.map(FileChannel.MapMode.READ_ONLY, 0, channel.size());
```

Once you have the buffer, you can read and write data using the methods of the `ByteBuffer` class and the `Buffer` superclass.

Buffers support both sequential and random data access. A buffer has a *position* that is advanced by `get` and `put` operations. For example, you can sequentially traverse all bytes in the buffer as

```
while (buffer.hasRemaining()) {
    byte b = buffer.get();
    . . .
}
```

Alternatively, you can use random access:

```
for (int i = 0; i < buffer.limit(); i++) {
    byte b = buffer.get(i);
    . . .
}
```

You can also read and write arrays of bytes with the methods

```
get(byte[] bytes)
get(byte[] bytes, int offset, int length)
```

Finally, there are methods

```
getInt            getChar
getLong           getFloat
getShort          getDouble
```

to read primitive-type values that are stored as *binary* values in the file. As we already mentioned, Java uses big-endian ordering for binary data. However, if you need to process a file containing binary numbers in little-endian order, simply call

```
buffer.order(ByteOrder.LITTLE_ENDIAN);
```

To find out the current byte order of a buffer, call

```
ByteOrder b = buffer.order();
```

To write numbers to a buffer, use one of the methods

```
putInt            putChar
putLong           putFloat
putShort          putDouble
```

At some point, and certainly when the channel is closed, these changes are written back to the file.

Listing 2.3 computes the 32-bit cyclic redundancy checksum (CRC32) of a file. That checksum is often used to determine whether a file has been corrupted. Corruption of a file makes it very likely that the checksum has changed. The `java.util.zip` package contains a class `CRC32` that computes the checksum of a sequence of bytes, using the following loop:

```
var crc = new CRC32();
while (more bytes) {
    crc.update(next byte);
}
long checksum = crc.getValue();
```

The details of the CRC computation are not important. We just use it as an example of a useful file operation. (In practice, you would read and update data in larger blocks, not a byte at a time. Then the speed differences are not as dramatic.)

Run the program as

```
java v2ch02.memoryMap.MemoryMapDemo filename
```

Listing 2.3 v2ch02/memoryMap/MemoryMapDemo.java

```
1  package v2ch02.memoryMap;
2
3  import module java.base;
```

```
4
5  /**
6   * This program computes the CRC checksum of a file in four ways.
7   */
8  public class MemoryMapDemo {
9      void main(String[] args) throws Exception {
10         Path filename = Path.of(args.length > 0 ? args[0]
11                 : IO.readln("Enter the path to a large file: "));
12         IO.println("Input Stream:");
13         long start = System.nanoTime();
14         long crcValue = checksumInputStream(filename);
15         long end = System.nanoTime();
16         IO.println(Long.toHexString(crcValue));
17         IO.println("%.3f seconds".formatted((end - start) * 1E-9));
18
19         IO.println("Buffered Input Stream:");
20         start = System.nanoTime();
21         crcValue = checksumBufferedInputStream(filename);
22         end = System.nanoTime();
23         IO.println(Long.toHexString(crcValue));
24         IO.println("%.3f seconds".formatted((end - start) * 1E-9));
25
26         IO.println("Random Access File:");
27         start = System.nanoTime();
28         crcValue = checksumRandomAccessFile(filename);
29         end = System.nanoTime();
30         IO.println(Long.toHexString(crcValue));
31         IO.println("%.3f seconds".formatted((end - start) * 1E-9));
32
33         IO.println("Mapped File:");
34         start = System.nanoTime();
35         crcValue = checksumMappedFile(filename);
36         end = System.nanoTime();
37         IO.println(Long.toHexString(crcValue));
38         IO.println("%.3f seconds".formatted((end - start) * 1E-9));
39     }
40
41     long checksumInputStream(Path filename) throws IOException {
42         try (InputStream in = Files.newInputStream(filename)) {
43             var crc = new CRC32();
44             boolean done = false;
45             while (!done) {
46                 int c = in.read();
47                 if (c == -1)
48                     done = true;
49                 else
50                     crc.update(c);
51             }
52             return crc.getValue();
53         }
54     }
55
56     long checksumBufferedInputStream(Path filename) throws IOException {
57         try (var in = new BufferedInputStream(Files.newInputStream(filename))) {
58             var crc = new CRC32();
59
60             boolean done = false;
```

```
            while (!done) {
                int c = in.read();
                if (c == -1)
                    done = true;
                else
                    crc.update(c);
            }
            return crc.getValue();
        }
    }

    long checksumRandomAccessFile(Path filename) throws IOException {
        try (var file = new RandomAccessFile(filename.toFile(), "r")) {
            long length = file.length();
            var crc = new CRC32();

            for (long p = 0; p < length; p++) {
                int c = file.readByte();
                crc.update(c);
            }
            return crc.getValue();
        }
    }

    long checksumMappedFile(Path filename) throws IOException {
        try (FileChannel channel = FileChannel.open(filename)) {
            var crc = new CRC32();
            int length = (int) channel.size();
            MappedByteBuffer buffer = channel.map(FileChannel.MapMode.READ_ONLY, 0, length);

            for (int p = 0; p < length; p++) {
                int c = buffer.get(p);
                crc.update(c);
            }
            return crc.getValue();
        }
    }
}
```

`java.io.FileInputStream` 1.0

- `FileChannel getChannel()` 1.4
  returns a channel for accessing this input stream.

`java.io.FileOutputStream` 1.0

- `FileChannel getChannel()` 1.4
  returns a channel for accessing this output stream.

`java.io.RandomAccessFile` 1.0

- `FileChannel getChannel()` 1.4
  returns a channel for accessing this file.

`java.nio.channels.FileChannel` 1.4

- `static FileChannel open(Path path, OpenOption... options)` 7
  opens a file channel for the given path. By default, the channel is opened for reading. The parameter `options` is one of the values `WRITE`, `APPEND`, `TRUNCATE_EXISTING`, `CREATE` in the `StandardOpenOption` enumeration.
- `MappedByteBuffer map(FileChannel.MapMode mode, long position, long size)`
  maps a region of the file to memory. The parameter `mode` is one of the constants `READ_ONLY`, `READ_WRITE`, or `PRIVATE` in the `FileChannel.MapMode` class.

`java.nio.Buffer` 1.4

- `boolean hasRemaining()`
  returns `true` if the current buffer position has not yet reached the buffer's limit position.
- `int limit()`
  returns the limit position of the buffer—that is, the first position at which no more values are available.

`java.nio.ByteBuffer` 1.4

- `byte get()`
  gets a byte from the current position and advances the current position to the next byte.
- `byte get(int index)`
  gets a byte from the specified index.
- `ByteBuffer put(byte b)`
  puts a byte at the current position and advances the current position to the next byte. Returns a reference to this buffer.
- `ByteBuffer put(int index, byte b)`
  puts a byte at the specified index. Returns a reference to this buffer.
- `ByteBuffer get(byte[] destination)`
- `ByteBuffer get(byte[] destination, int offset, int length)`
  fill a byte array, or a region of a byte array, with bytes from the buffer, and advance the current position by the number of bytes read. If not enough bytes remain in the buffer, then no bytes are read, and a `BufferUnderflowException` is thrown. Return a reference to this buffer.
- `ByteBuffer put(byte[] source)`
- `ByteBuffer put(byte[] source, int offset, int length)`
  put all bytes from a byte array, or the bytes from a region of a byte array, into the buffer, and advance the current position by the number of bytes read. If not enough bytes remain in the buffer, then no bytes are written, and a `BufferOverflowException` is thrown. Return a reference to this buffer.
- `Xxx getXxx()`
- `Xxx getXxx(int index)`
- `ByteBuffer putXxx(Xxx value)`
- `ByteBuffer putXxx(int index, Xxx value)`
  get or put a binary number. Xxx is one of `Int`, `Long`, `Short`, `Char`, `Float`, or `Double`.
- `ByteBuffer order(ByteOrder order)`
- `ByteOrder order()`
  set or get the byte order. The value for `order` is one of the constants `BIG_ENDIAN` or `LITTLE_ENDIAN` of the `ByteOrder` class.

- `static ByteBuffer allocate(int capacity)`
  constructs a buffer with the given capacity.
- `static ByteBuffer wrap(byte[] values)`
  constructs a buffer that is backed by the given array.
- `CharBuffer asCharBuffer()`
  constructs a character buffer that is backed by this buffer. Changes to the character buffer will show up in this buffer, but the character buffer has its own position, limit, and mark.

`java.nio.CharBuffer` 1.4

- `char get()`
- `CharBuffer get(char[] destination)`
- `CharBuffer get(char[] destination, int offset, int length)`
  get one `char` value, or a range of `char` values, starting at the buffer's position and moving the position past the characters that were read. The last two methods return `this`.
- `CharBuffer put(char c)`
- `CharBuffer put(char[] source)`
- `CharBuffer put(char[] source, int offset, int length)`
- `CharBuffer put(String source)`
- `CharBuffer put(CharBuffer source)`
  put one `char` value, or a range of `char` values, starting at the buffer's position and advancing the position past the characters that were written. When reading from a `CharBuffer`, all remaining characters are read. All methods return `this`.

2.4.2. The Buffer Data Structure

When you use memory mapping, you make a single buffer that spans the entire file or the area of the file that you're interested in. You can also use buffers to read and write more modest chunks of information.

In this section, we briefly describe the basic operations on `Buffer` objects. A buffer is an array of values of the same type. The `Buffer` class is an abstract class with concrete subclasses `ByteBuffer`, `CharBuffer`, `DoubleBuffer`, `FloatBuffer`, `IntBuffer`, `LongBuffer`, and `ShortBuffer`.



Note: The `StringBuffer` class is not related to these buffers.

In practice, you will most commonly use `ByteBuffer` and `CharBuffer`. As shown in **#ch01fig11**, a buffer has

- A *capacity* that never changes
- A *position* at which the next value is read or written
- A *limit* beyond which reading and writing are meaningless
- Optionally, a *mark* for repeating a read or write operation

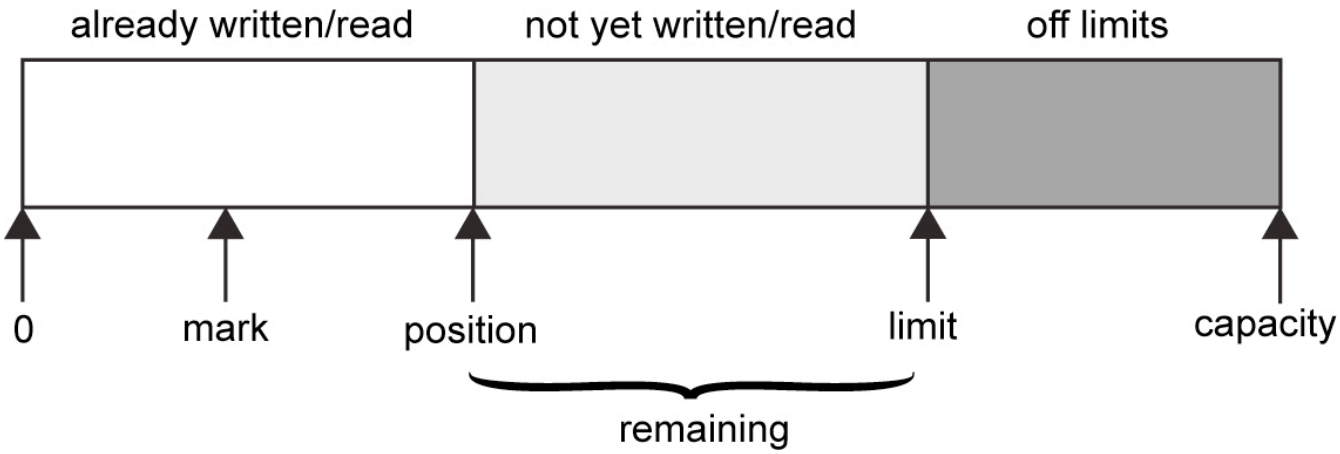


Figure 2.5: A buffer

These values fulfill the condition

$0 \leq \text{mark} \leq \text{position} \leq \text{limit} \leq \text{capacity}$

The principal purpose of a buffer is a sequence of "fill, then consume" cycles. At the outset, the buffer's position is 0 and the limit is the capacity. Keep filling the buffer by calling `put` or by reading from a channel.

When you run out of data or reach the capacity, it is time to switch to consuming the contents. Call `flip` to set the limit to the current position and the position to `0`.

Now keep consuming, by calling `get` or writing to a channel, while the `remaining` method (which computes *limit – position*) yields a positive value.

When you have consumed all values in the buffer, call `compact` to prepare the buffer for filling it again. Any unconsumed data is copied to the beginning of the buffer. The position is set to follow the copied data, and the limit is set to the capacity.

If you want to reread the buffer, use `rewind` or `mark`/`reset` (see the API notes for details).

To obtain a buffer, call a static method such as `ByteBuffer.allocate` or `ByteBuffer.wrap`.

This loop uses a buffer to transfer data from one channel to another:

```
ByteBuffer buffer = ByteBuffer.allocate(BUFFER_SIZE);
boolean doneReading = false;
boolean doneWriting = false;
while (!(doneReading && doneWriting)) {
    if (sourceChannel.read(buffer) == -1) doneReading = true;
    buffer.flip();
    destinationChannel.write(buffer);
    if (buffer.remaining() == 0) doneWriting = true;
    buffer.compact();
}
```

`java.nio.Buffer` 1.4

- `Buffer flip()`
  prepares this buffer for reading after writing, by setting the limit to the position and the position to `0`; returns `this`.
- `Buffer rewind()`
  prepares this buffer for rereading the same values by setting the position to `0` and leaving the limit unchanged; returns `this`.
- `Buffer clear()`
  prepares this buffer for writing by setting the position to `0` and the limit to the capacity; returns `this`.
- `Buffer mark()`
  sets the mark of this buffer to the position; returns `this`.
- `Buffer reset()`
  sets the position of this buffer to the mark, thus allowing the marked portion to be read or written again; returns `this`.
- `int remaining()`
  returns the remaining number of readable or writable values—that is, the difference between the limit and position.

`java.nio.ByteBuffer` 1.4

- `Buffer compact()`
  copies any unconsumed bytes to the beginning of the buffer, sets the position to the following byte, and the limit to the capacity; returns `this`.

`java.nio.ReadableByteChannel` 1.4

- `int read(ByteBuffer dst)`
  fills the remaining part of the buffer with available bytes, and returns the number of transferred bytes, or -1 if the channel has reached its end.

`java.nio.WritableByteChannel` 1.4

- `int write(ByteBuffer src)`
  attempts to consume the remaining bytes from the buffer, and returns the number of transferred bytes.

2.5. File Locking

When multiple simultaneously executing programs need to modify the same file, they need to communicate in some way, or the file can easily become damaged. File locks can solve this problem. A file lock controls access to a file or a range of bytes within a file.

Suppose your application saves a file with system-wide preferences. If a user invokes two instances of the application, it could happen that both of them want to write the file at the same time. In that situation, the first instance should lock the file. When the second instance finds the file locked, it can decide to wait until the file is unlocked or simply skip the writing process.

To lock a file, call either the `lock` or `tryLock` methods of the `FileChannel` class.

```
FileChannel channel = FileChannel.open(path);
FileLock lock = channel.lock();
```

or

```
FileLock lock = channel.tryLock();
```

The first call blocks until the lock becomes available. The second call returns immediately, either with the lock or with `null` if the lock is not available. The file remains locked until the channel is closed or the `release` method is invoked on the lock.

You can also lock a portion of the file with the call

```
FileLock lock(long start, long size, boolean shared)
```

or

```
FileLock tryLock(long start, long size, boolean shared)
```

The `shared` flag is `false` to lock the file for both reading and writing. It is `true` for a *shared* lock, which allows multiple processes to read from the file, while preventing any process from acquiring an exclusive lock. Not all operating systems support shared locks. You may get an exclusive lock even if you just asked for a shared one. Call the `isShared` method of the `FileLock` class to find out which kind you have.



Note: If you lock the tail portion of a file and the file subsequently grows beyond the locked portion, the additional area is not locked. To lock all bytes, use a size of `Long.MAX_VALUE`.

Be sure to unlock the lock when you are done. As always, this is best done with a `try`-with-resources statement:

```
try (FileLock _ = channel.lock()) {
    access the locked file or segment
}
```

Keep in mind that file locking is system-dependent. Here are some points to watch for:

- On some systems, including Linux, file locking is merely *advisory*. If an application fails to get a lock, it may still write to a file that another application has currently locked.
- On some systems, you cannot simultaneously lock a file and map it into memory.
- File locks are global to the Java virtual machine. Concurrent threads can't each acquire a lock on the same file. The `lock` and `tryLock` methods will throw an `OverlappingFileLockException` if another thread already holds a lock on the same file or an overlapping range.
- On some systems, closing a channel releases all locks on the underlying file. You should therefore avoid multiple channels on the same locked file.

- Locking files on a networked file system is highly system-dependent and should probably be avoided.
- File locking is only guaranteed for Java programs. Programs written in other languages may use different and incompatible locking mechanisms.

The program in **Listing 2.4** shows a simple example of file locking. Multiple users want to register their passwords in a common file. The program reads in the file, adds the username and hashed password, and writes the resulting file. It is important that this update is atomic. Otherwise, the password file can be corrupted if two users update it concurrently.

See **Chapter 9** for more information about password hashing. It has nothing to do with file locking, but I didn't want to show a sample program that saves unhashed passwords.

The program shows the limitations of the file locking API. It is tied to the `FileChannel` API, which is based on the reading and writing of byte buffers. The program has to turn those byte buffers into readers and writers.

An alternative would be to use a `RandomAccessFile` whose `getChannel` method yields a channel to the file. But it is equally inconvenient. One needs to read the file into a byte array and prepare a byte array to write the modified file.

Listing 2.4 v2ch02/fileLock/FileLockDemo.java

```
1  package v2ch02.fileLock;
2
3  import module java.base;
4
5  /**
6   * This program writes user names and passwords to the file passwords.properties. The file is
7   * atomically updated. With Linux, try
8   * for f in $(seq 1 100) ; do (java fileLock.FileLockDemo user$f secret$f &) ; done
9   * When all Java processes have finished, then passwords.properties
10  * has an entry for each of them.
11  */
12
13 public class FileLockDemo {
14     void main(String[] args) throws Exception {
15         Path path = Path.of("password.properties");
16         FileChannel channel = FileChannel.open(path, StandardOpenOption.READ,
17                 StandardOpenOption.WRITE, StandardOpenOption.CREATE);
18         try (FileLock _ = channel.lock()) {
19             // Read bytes from file
20             ByteBuffer readBuffer = ByteBuffer.allocate((int) channel.size());
21             channel.read(readBuffer);
22
23             // Read props from bytes
24             Reader in = new StringReader(new String(readBuffer.array()));
25             Properties props = new Properties();
26             props.load(in);
27
28             // Get username, password from args or user input
29             String username;
30             char[] password;
31             if (args.length >= 2) {
32                 username = args[0];
33                 password = args[1].toCharArray();
```

```
            }
            else {
                Console console = System.console();
                username = console.readLine("User name: ");
                password = console.readPassword("Password: ");
            }

            // Hash password
            final int ITERATIONS = 210_000;
            final int SALT_BYTES = 16;
            final int HASH_BYTES = 32;
            SecureRandom random = new SecureRandom();
            byte[] salt = new byte[SALT_BYTES];
            random.nextBytes(salt);

            var spec = new PBEKeySpec(password, salt, ITERATIONS, 8 * HASH_BYTES);
            SecretKeyFactory skf = SecretKeyFactory.getInstance("PBKDF2WithHmacSHA256");
            byte[] hash = skf.generateSecret(spec).getEncoded();
            Base64.Encoder encoder = Base64.getEncoder();

            // Put username and hashed password into props
            props.put(username, ITERATIONS + "|" + encoder.encodeToString(salt) + "|"
                    + encoder.encodeToString(hash));

            // Write props to bytes
            StringWriter out = new StringWriter();
            props.store(out, null);
            byte[] outBytes = out.toString().getBytes();

            // Write bytes to file
            channel.truncate(0);
            ByteBuffer writeBuffer = ByteBuffer.wrap(outBytes);
            channel.write(writeBuffer);
        }
    }
}
```

`java.nio.channels.FileChannel` 1.4

- `FileLock lock()`
  acquires an exclusive lock on the entire file. This method blocks until the lock is acquired.
- `FileLock tryLock()`
  acquires an exclusive lock on the entire file, or returns `null` if the lock cannot be acquired.
- `FileLock lock(long position, long size, boolean shared)`
- `FileLock tryLock(long position, long size, boolean shared)`
  acquire a lock on a region of the file. The first method blocks until the lock is acquired, and the second method returns `null` if the lock cannot be acquired. The parameter `shared` is `true` for a shared lock, `false` for an exclusive lock.
- `FileChannel truncate(long size)`
  truncates the file to the given size if it is larger.

`java.nio.channels.FileLock` 1.4

- `void close()` 7
  releases this lock.

`java.io.RandomAccessFile` 1.0

- `FileChannel getChannel()` 1.4
  yields the channel for this random access file.

2.6. Object Input/Output Streams and Serialization

In **Section 2.1.7** and **Section 2.2.2**, you have seen how to store `Employee` objects in text or binary format. However, objects that you create in an object-oriented program are rarely all of the same type. For example, you might have an array called `staff` that is nominally an array of `Employee` records but contains objects that are actually instances of a subclass such as `Manager`.

It is certainly possible to come up with a data format that allows you to store such polymorphic collections—but, fortunately, we don't have to. The Java language supports a very general mechanism, called *object serialization*, that makes it possible to write any object to an output stream and read it again later. (You will see in the following sections where the term "serialization" comes from.)

2.6.1. Saving and Loading Serializable Objects

To save object data, you first need to open an `ObjectOutputStream` object:

```
var out = new ObjectOutputStream(new FileOutputStream("employee.dat"));
```

Now, to save an object, simply use the `writeObject` method of the `ObjectOutputStream` class as in the following fragment:

```
var harry = new Employee("Harry Hacker", 50000, 1989, 10, 1);
var boss = new Manager("Carl Cracker", 80000, 1987, 12, 15);
out.writeObject(harry);
out.writeObject(boss);
```

To read the objects back in, first get an `ObjectInputStream` object:

```
var in = new ObjectInputStream(new FileInputStream("employee.dat"));
```

Then, retrieve the objects in the same order in which they were written, using the `readObject` method:

```
var e1 = (Employee) in.readObject();
var e2 = (Employee) in.readObject();
```

There is, however, one change you need to make to any class that you want to save to an output stream and restore from an object input stream. The class must implement the `Serializable` interface:

```
class Employee implements Serializable { . . . }
```

The `Serializable` interface has no methods, so you don't need to change your classes in any way. In this regard, it is similar to the `Cloneable` interface that we discussed in **Chapter 6 of Volume I**. However, to make a class cloneable, you still had to override the `clone` method of the `Object` class. To make a class serializable, you do not need to do anything else.



Note: You can write and read only *objects* with the `writeObject/readObject` methods. For primitive type values, use methods such as `writeInt/readInt` or `writeDouble/readDouble`. (The object input/output stream classes implement the `DataInput/DataOutput` interfaces.)

Behind the scenes, an `ObjectOutputStream` looks at all the fields of the objects and saves their contents. For example, when writing an `Employee` object, the name, date, and salary fields are written to the output stream.

However, there is one important situation to consider: What happens when one object is shared by several objects as part of their state?

To illustrate the problem, let us make a slight modification to the `Manager` class. Let's assume, nostalgically, that each manager has a secretary:

```
class Manager extends Employee {
    private Employee secretary;
    . . .
}
```

Each `Manager` object now contains a reference to an `Employee` object that describes the secretary. Of course, two managers can share the same secretary, as is the case in **#ch01fig06** and the following code:

```
var harry = new Employee("Harry Hacker", . . .);
var carl = new Manager("Carl Cracker", . . .);
carl.setSecretary(harry);
var tony = new Manager("Tony Tester", . . .);
tony.setSecretary(harry);
```

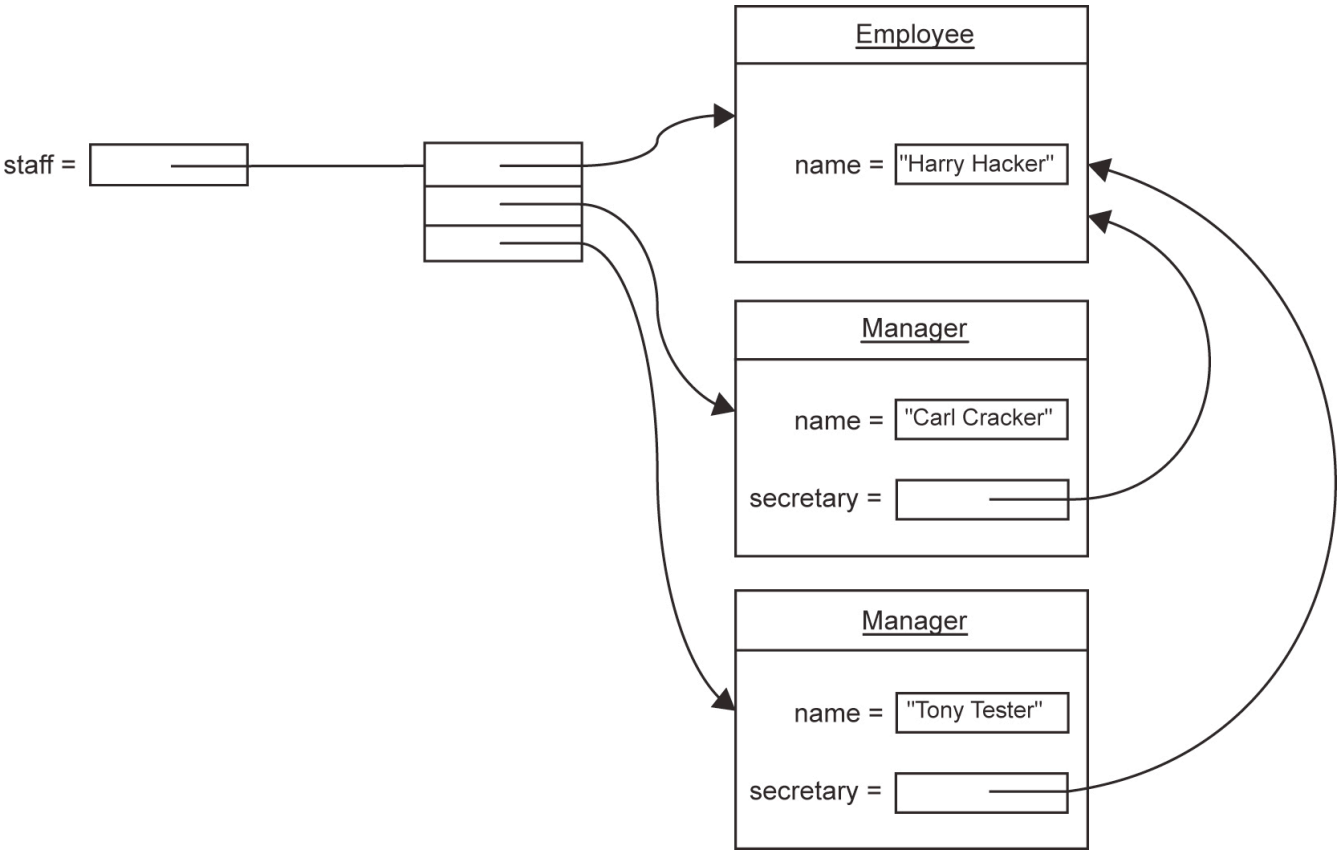


Figure 2.6: Two managers can share a mutual employee.

Saving such a network of objects is a challenge. Of course, we cannot save and restore the memory addresses for the secretary objects. When an object is reloaded, it will likely occupy a completely different memory address than it originally did.

Instead, each object is saved with a *serial number*—hence the name *object serialization* for this mechanism. Here is the algorithm:

1. Associate a serial number with each object reference that you encounter (as shown in **#ch01fig07**).

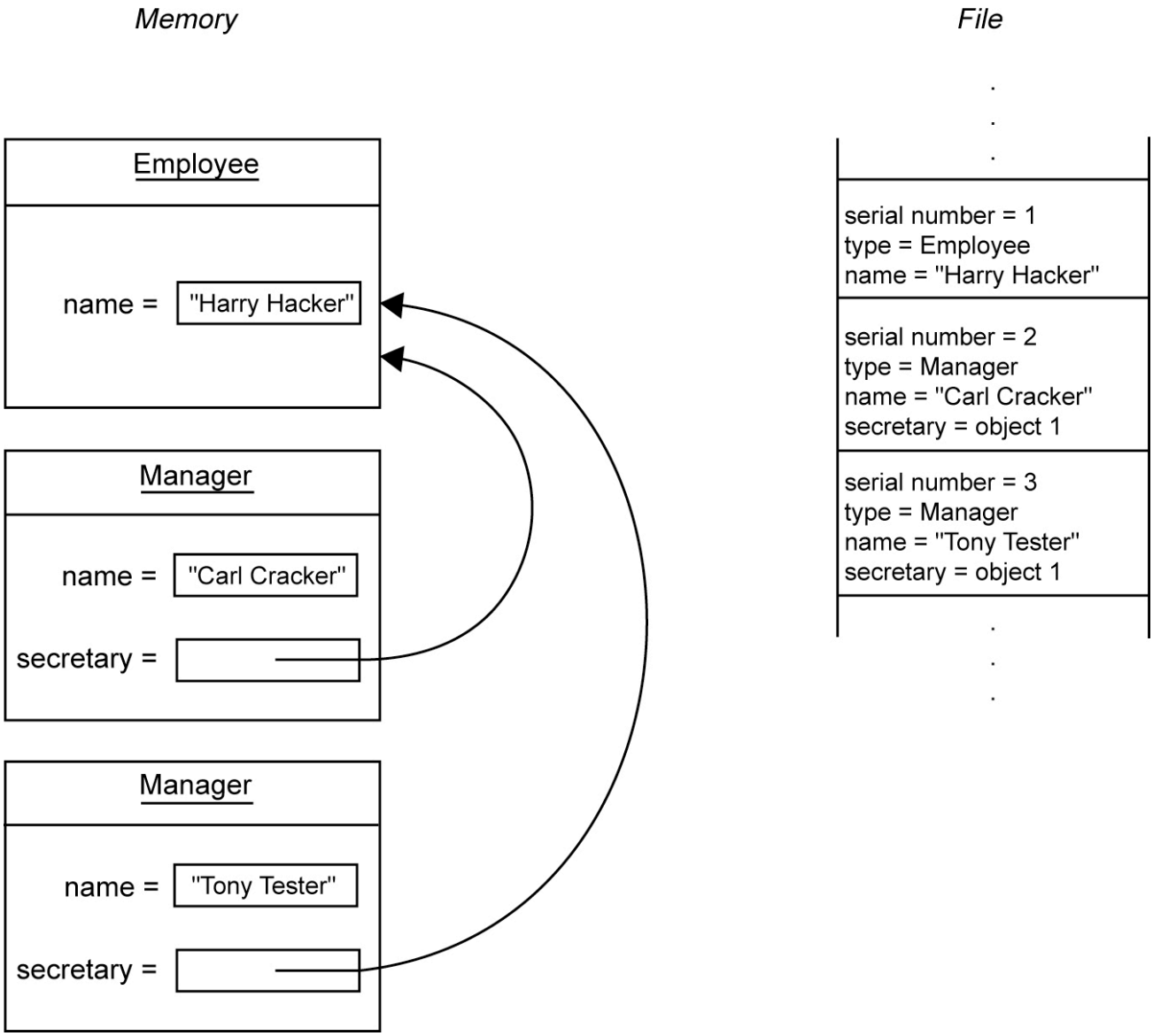


Figure 2.7: An example of object serialization

2. When encountering an object reference for the first time, save the object data to the output stream.
3. If it has been saved previously, just write "same as the previously saved object with serial number x."

When reading the objects back, the procedure is reversed.

1. When an object is specified in an object input stream for the first time, construct it, initialize it with the stream data, and remember the association between the serial number and the object reference.
2. When the tag "same as the previously saved object with serial number x" is encountered, retrieve the object reference for the sequence number.

Note: In this chapter, we will use serialization to save a collection of objects to a disk file and retrieve it exactly as we stored it. Another very important application is the transmittal of a collection of objects across a network connection to another computer. Just as raw memory addresses are meaningless in a file, they are also meaningless when you communicate with a different processor. By replacing memory addresses with serial numbers, serialization permits the transport of object collections from one machine to another.

Note: If the superclass of a serializable class is not serializable, it must have an accessible no-argument constructor. Consider this example:

```
class Person // Not serializable
class Employee extends Person implements Serializable
```

When an `Employee` object is deserialized, its instance fields are read from the object input stream, but the `Person` instance fields are set by the `Person` constructor.

Listing 2.5 is a program that saves and reloads a network of `Employee` and `Manager` objects (some of which share the same employee as a secretary). Note that the secretary object is unique after reloading—when `newStaff[1]` gets a raise, that is reflected in the `secretary` fields of the managers.

Listing 2.5 v2ch02/serial/ObjectStreamDemo.java

```
1  package v2ch02.serial;
2
3  import module java.base;
4
5  class ObjectStreamDemo {
6      void main() throws Exception {
7          var harry = new Employee("Harry Hacker", 50000, 1989, 10, 1);
8          var carl = new Manager("Carl Cracker", 80000, 1987, 12, 15);
9          carl.setSecretary(harry);
10         var tony = new Manager("Tony Tester", 40000, 1990, 3, 15);
11         tony.setSecretary(harry);
12
13         var staff = new Employee[3];
14
15         staff[0] = carl;
16         staff[1] = harry;
17         staff[2] = tony;
18
19         // save all employee records to the file employee.ser
20         try (var out = new ObjectOutputStream(new FileOutputStream("employee.ser"))) {
21             out.writeObject(staff);
22         }
23
24         try (var in = new ObjectInputStream(new FileInputStream("employee.ser"))) {
25             // retrieve all records into a new array
26
27             var newStaff = (Employee[]) in.readObject();
28
29             // raise secretary's salary
30             newStaff[1].raiseSalary(10);
31
32             // print the newly read employee records
33             for (Employee e : newStaff)
34                 IO.println(e);
35         }
36     }
37 }
```

`java.io.ObjectOutputStream` 1.1

- `ObjectOutputStream(OutputStream out)`
  creates an `ObjectOutputStream` so you can write objects to the specified `OutputStream`.
- `void writeObject(Object obj)`
  writes the specified object to the `ObjectOutputStream`. This method saves the class of the object, the signature of the class, and the values of any nonstatic, nontransient fields of the class and its superclasses.

`java.io.ObjectInputStream` 1.1

- `ObjectInputStream(InputStream in)`
  creates an `ObjectInputStream` to read back object information from the specified `InputStream`.
- `Object readObject()`
  reads an object from the `ObjectInputStream`. In particular, this method reads back the class of the object, the signature of the class, and the values of the nontransient and nonstatic fields of the class and all its superclasses. It does deserializing so that multiple object references can be recovered.

2.6.2. Understanding the Object Serialization File Format

Object serialization saves object data in a particular file format. Of course, you can use the `writeObject/readObject` methods without having to know the exact sequence of bytes that represents objects in a file. Nonetheless, we found studying the data format extremely helpful for gaining insight into the object serialization process. As the details are somewhat technical, feel free to skip this section if you are not interested in the implementation.

Every file begins with the two-byte “magic number”

```
AC ED
```

followed by the version number of the object serialization format, which is currently

```
00 05
```

(We use hexadecimal numbers throughout this section to denote bytes.) Then, it contains a sequence of objects, in the order in which they were saved.

String objects are saved as

- `74`
- Two-byte length
- Characters in the “modified UTF-8” format

For example, the string “Harry” is saved as

```
74 00 05 Harry
```

When an object is saved, the class of that object must be saved as well. The class descriptor has the following format:

- `72`
- 2-byte length of class name
- Class name
- 8-byte fingerprint
- 1-byte flag
- 2-byte count of instance field descriptors
- Instance field descriptors
- `78` (end marker)
- Superclass descriptor (`70` if none)

The fingerprint is obtained by ordering the descriptions of the class, superclass, interfaces, field types, and method signatures in a canonical way, and then applying the Secure Hash Algorithm version 1 (SHA-1) to that data.

SHA-1 gives a "fingerprint" of a block of information. This fingerprint is always a 20-byte data packet, regardless of the size of the original data. It is created by a clever sequence of bit operations on the data that makes it extremely likely that the fingerprint will change if the information is altered in any way. The serialization mechanism uses only the first eight bytes of the SHA-1 code as a class fingerprint. It is still very likely that the class fingerprint will change if the instance fields or methods change.

When reading an object, its fingerprint is compared against the current fingerprint of the class. If they don't match, it means the class definition has changed after the object was written, and an exception is generated. Of course, in practice, classes do evolve, and it might be necessary for a program to read in older versions of objects. We will discuss this in **Section 2.6.7**.

The flag byte is composed of the bit masks, defined in `java.io.ObjectStreamConstants`, shown in **Table 2.7**.

Table 2.7: Bit Masks

Name	Value	Description
`SC_WRITE_METHOD`	1	Class defines `writeObject` method.
`SC_SERIALIZABLE`	2	Class implements `Serializable`.
`SC_EXTERNALIZABLE`	4	Class implements `Externalizable`.
`SC_BLOCK_DATA`	8	Opaque data written as "block data" (default since JDK 1.2).
`SC_ENUM`	16	Class is an enumeration.

The classes that we write implement the `Serializable` interface and will have a flag value of `02`. The serializable `java.util.Date` class has a flag of `03` because it defines its own `writeObject` method (see **Section 2.6.4**). We discuss the `Externalizable` interface in **Section 2.6.5**.

Each instance field descriptor has the format:

- 1-byte type code
- 2-byte length of field name
- Field name
- Class name (if the field is an object)

The type code is one of the following:

`B`	`byte`
`C`	`char`
`D`	`double`
`F`	`float`
`I`	`int`
`J`	`long`
`L`	object
`S`	`short`
`Z`	`boolean`
`[`	array

When the type code is `L`, the field name is followed by the field type. Class and field name strings do not start with the string code `74`, but field types do. Field types use a slightly different encoding of their names—namely, the format used by native methods.

For example, the `salary` field of the `Employee` class is encoded as

```
D 00 06 salary
```

Here is the complete class descriptor of the `Employee` class, contained in the `serial` package in our example program:

```
72 00 0F serial.Employee
```

`74 1E C8 E6 C3 F8 B8 77 02`	Fingerprint and flags

`00 03`	Number of instance fields
`D 00 06 salary`	Instance field type and name
`L 00 07 hireDay`	Instance field type and name
`74 00 10 Ljava/util/Date;`	Instance field class name: `java.util.Date`
`L 00 04 name`	Instance field type and name
`74 00 12 Ljava/lang/String;`	Instance field class name: `java.lang.String`
`78`	End marker
`70`	No superclass

These descriptors are fairly long. If the *same* class descriptor is needed again in the file, an abbreviated form is used:

- `71`
- 4-byte serial number

The serial number refers to the previous explicit class descriptor. We discuss the numbering scheme later.

An object is stored as

- `73`

- Class descriptor
- Object data

For example, here is how the instance fields of an `Employee` object are stored:

`40 E8 6A 00 00 00 00 00`	`salary` field value: `double`
`73`	`hireDay` field value: new object
`71 00 7E 00 08`	Existing class `java.util.Date`
`77 08 00 00 00 91 19 97 3D 80 78`	External storage (details later)
`74 00 0C Harry Hacker`	`name` field value: `String`

As you can see, the data file contains enough information to restore the `Employee` object. (In this example, we store the hireday as a `Date` instead of a `LocalDate`, since the `LocalDate` class has a more complex serialization.)



Caution: As you just saw, the object stream contains the names of the classes, superclasses, and fields of all serialized objects. For inner classes, some of these names are synthesized by the compiler, and the naming convention may change from one Java implementation to another. If that occurs, deserialization fails. Therefore, there is a small risk with serializing inner classes if you intend to have them deserialized with a different implementation.

Static inner classes, including inner enumerations and records, are safe to serialize.

Arrays are saved in the following format:

- `75`
- Class descriptor
- 4-byte number of elements
- Elements

The array class name in the class descriptor is in the same format as that used by native methods (which is slightly different from the format used by class names in other class descriptors). In this format, class names start with an `L` and end with a semicolon.

For example, an array of three `Employee` objects starts out like this:

`75`	Array
`72 00 0B [LEmployee;`	New class, string length, class name `Employee[]`
`40 05 D6 7E 3B BB F2 50 02`	Fingerprint and flags
`00 00`	Number of instance fields
`78`	End marker
`70`	No superclass
`00 00 00 03`	Number of array entries

Note that the fingerprint for an array of `Employee` objects is different from a fingerprint of the `Employee` class itself.

All objects (including arrays and strings) and all class descriptors are given serial numbers as they are saved in the output file. The numbers start at `00 7E 00 00`.

We already saw that a full class descriptor for any given class occurs only once. Subsequent descriptors refer to it. For example, in our previous example, a repeated reference to the `Date` class was coded as

```
71 00 7E 00 08
```

The same mechanism is used for objects. If a reference to a previously saved object is written, it is saved in exactly the same way—that is, `71` followed by the serial number. It is always clear from the context whether a particular serial reference denotes a class descriptor or an object.

Some classes (such as the `Date` class) store object state in an opaque binary block, encoded as

- `77` (short block) or `7A` (long block)
- 1-byte or 4-byte block length
- Block contents
- `78`

Finally, a null reference is stored as

```
70
```

Here is the commented output of the `ObjectStreamDemo` program of the preceding section. Run the program, look at a hex dump of its data file `employee.ser`, and compare it with the commented listing. The most important lines are toward the end of the output. They show a reference to a previously saved object.

`AC ED 00 05`	File header
`75`	Array `staff` (serial #1)
`72 00 12 [Lserial.Employee;`	New class, string length, class name `Employee[]` (serial #0)
`40 05 D6 7E 3B BB F2 50 02`	Fingerprint and flags
`00 00`	Number of instance fields

`78`	End marker
`70`	No superclass
`00 00 00 03`	Number of array entries
`73`	`staff[0]` —new object (serial #7)
`72 00 0E serial.Manager`	New class, string length, class name (serial #2)
`2F FF 1F 1D E8 7A A9 74 02`	Fingerprint and flags
`00 01`	Number of instance fields
`L 00 09 secretary`	Instance field type and name
`74 00 11 Lserial/Employee;`	Instance field class name: `String` (serial #3)
`78`	End marker

`72 00 0F serial.Employee`	Superclass: new class, string length, class name (serial #4)
`74 1E C8 E6 C3 F8 B8 77 02`	Fingerprint and flags
`00 03`	Number of instance fields
`D 00 06 salary`	Instance field type and name
`L 00 07 hireDay`	Instance field type and name
`74 00 10 Ljava/util/Date;`	Instance field class name: `String` (serial #5)
`L 00 04 name`	Instance field type and name
`74 00 12 Ljava/lang/String;`	Instance field class name: `String` (serial #6)
`78`	End marker
`70`	No superclass

`40 F3 88 00 00 00 00 00`	`salary` field value: `double`
`73`	`hireDay` field value: new object (serial #9)
`72 00 0E java.util.Date`	New class, string length, class name (serial #8)
`68 6A 81 01 4B 59 74 19 03`	Fingerprint and flags
`00 00`	No instance fields
`78`	End marker
`70`	No superclass
`77 08`	External storage, number of bytes
`00 00 00 83 E7 4B 7D 80`	Opaque bytes
`78`	End marker

`74 00 0C Carl Cracker`	`name` field value: `String` (serial #10)
`73`	`secretary` field value: new object (serial #11)
`71 00 7E 00 04`	existing class (use serial #4)
`40 E8 6A 00 00 00 00 00`	`salary` field value: `double`
`73`	`hireDay` field value: new object (serial #12)
`71 00 7E 00 08`	Existing class (use serial #8)
`77 08`	External storage, number of bytes
`00 00 00 91 19 97 3D 80`	Opaque bytes
`78`	End marker
`74 00 0C Harry Hacker`	`name` field value: `String` (serial #13)

`71 00 7E 00 0B`	`staff[1]` : existing object (use serial #11)
`73`	`staff[2]` : new object (serial #14)
`71 00 7E 00 02`	Existing class (use serial #2)
`40 E3 88 00 00 00 00 00`	`salary` field value: `double`
`73`	`hireDay` field value: new object (serial #15)
`71 00 7E 00 08`	Existing class (use serial #8)
`77 08`	External storage, number of bytes
`00 00 00 94 6B 50 89 80`	Opaque bytes
`78`	End marker
`74 00 0B Tony Tester`	`name` field value: `String` (serial #16)

`71 00 7E 00 0B`	`secretary` field value: existing object (use serial #11)

Of course, studying these codes can be about as exciting as reading a phone book. It is not important to know the exact file format (unless you are trying to create an evil effect by modifying the data), but it is still instructive to know that the serialized format has a detailed description of all the objects it contains, with sufficient detail to allow reconstruction of both objects and arrays of objects.

What you should remember is this:

- The serialized format contains the types and instance fields of all objects.
- Each object is assigned a serial number.
- Repeated occurrences of the same object are stored as references to that serial number.

2.6.3. Transient Fields

Certain fields should never be serialized—for example, database connections that are meaningless when an object is reconstituted. Also, when an object keeps a cache of values, it might be better to drop the cache and recompute it instead of storing it.

To prevent a field from being serialized, tag it with the `transient` modifier. Always mark fields as `transient` if they hold instances of nonserializable classes. Transient fields are simply skipped when objects are serialized.

2.6.4. The `readObject` and `writeObject` Methods

In rare cases, you need to tweak the serialization mechanism. A serializable class can add any desired action to the default read and write behavior by defining methods with the signature

```
@Serial private void readObject(ObjectInputStream in)
        throws IOException, ClassNotFoundException;
@Serial private void writeObject(ObjectOutputStream out)
        throws IOException;
```

Then, the object headers continue to be written as usual, but the fields are no longer automatically serialized. Instead, these methods are called.

Note the `@Serial` annotation. The methods for tweaking serialization don't belong to interfaces. Therefore, you can't use the `@Override` annotation to have the compiler check the method declarations. The `@Serial` annotation is meant to enable the same checking for serialization methods. Up to Java 21, the `javac` compiler doesn't do such checking, but that might change in the future. The IntelliJ IDE uses the annotation.

Here is a typical example for customization. A number of classes in the `java.awt.geom` package, such as `Point2D.Double`, are not serializable. Now, suppose you want to serialize a class `LabeledPoint` that stores a `String` and a `Point2D.Double`. First, you need to mark the `Point2D.Double` field as `transient` to avoid a `NotSerializableException`.

```
public class LabeledPoint implements Serializable {
    private String label;
    private transient Point2D.Double point;
    . . .
}
```

In the `writeObject` method, first write the object descriptor and the `String` field, `label`, by calling the `defaultWriteObject` method. This is a special method of the `ObjectOutputStream` class that can only be called from within a `writeObject` method of a serializable class. Then we write the point coordinates, using the standard `DataOutput` calls.

```
private void writeObject(ObjectOutputStream out) throws IOException {
    out.defaultWriteObject();
    out.writeDouble(point.getX());
    out.writeDouble(point.getY());
}
```

In the `readObject` method, we reverse the process:

```
private void readObject(ObjectInputStream in)
        throws IOException, ClassNotFoundException {
    in.defaultReadObject();
    double x = in.readDouble();
    double y = in.readDouble();
    point = new Point2D.Double(x, y);
}
```

Another example is the `HashSet` class that supplies its own `readObject` and `writeObject` methods. Instead of saving the internal structure of the hash table, the `writeObject` method simply saves the capacity, load factor, size, and elements. The `readObject` method reads back the capacity and load factor, constructs a new table, and inserts the elements.

The `readObject` and `writeObject` methods only need to save and load their data. They do not concern themselves with superclass data or any other class information.

The `Date` class uses this approach. Its `writeObject` method saves the milliseconds since the "epoch" (January 1, 1970). The data structure that caches calendar data is not saved. For that reason, the serialized `Date` instances in **Section 2.6.2** consist of an opaque data block.



Caution: Just like a constructor, the `readObject` method operates on partially initialized objects. If you call a non-final method inside `readObject` that is overridden in a subclass, it may access uninitialized data.



Note: If a serializable class defines a field

```
@Serial private static final ObjectStreamField[] serialPersistentFields
```

then serialization uses those field descriptors instead of the non-transient non-static fields. There is also an API for setting the field values before serialization, or reading them after deserialization. This is useful for preserving a legacy layout after a class has evolved. For example, the `BigDecimal` class uses this mechanism to serialize its instances in a format that no longer reflects the instance fields.

2.6.5. The `readExternal` and `writeExternal` Methods

Instead of letting the serialization mechanism save and restore object data, a class can define its own mechanism. For example, you can encrypt the data or use a format that is more efficient than the serialization format.

To do this, a class must implement the `Externalizable` interface. This, in turn, requires it to define two methods:

```
public void readExternal(ObjectInput in)
        throws IOException, ClassNotFoundException;
public void writeExternal(ObjectOutput out) throws IOException;
```

Unlike the `readObject` and `writeObject` methods, these methods are fully responsible for saving and restoring the entire object, *including the superclass data*. When writing an object, the serialization mechanism merely records the class of the object in the output stream. When reading an externalizable object, the object input stream creates an object with the no-argument constructor and then calls the `readExternal` method.

In this example, the `LabeledPixel` class extends the serializable `Point` class, but it takes over the serialization of the class and superclass. The fields of the object are not stored in the standard serialization format. Instead, the data are placed in an opaque block.

```
public class LabeledPixel extends Point implements Externalizable {
    private String label;

    public LabeledPixel() {} // required for externalizable class

    public void writeExternal(ObjectOutput out)
            throws IOException {
        out.writeInt((int) getX());
        out.writeInt((int) getY());
       out.writeUTF(label);
    }

    public void readExternal(ObjectInput in)
            throws IOException, ClassNotFoundException {
       int x = in.readInt();
       int y = in.readInt();
       setLocation(x, y);
       label = in.readUTF();
    }
    . . .
}
```



Note: The `readExternal` and `writeExternal` methods should not be annotated with `@Serial`. Since they are defined in the `Externalizable` interface, you can simply annotate them with `@Override`.



Caution: Unlike the `readObject` and `writeObject` methods, which are private and can only be called by the serialization mechanism, the `readExternal` and `writeExternal` methods are public. In particular, `readExternal` potentially permits modification of the state of an existing object.

2.6.6. The `readResolve` and `writeReplace` Methods

We take it for granted that objects can only be constructed with the constructor. However, a deserialized object is *not constructed*. Its fields are simply restored from an object stream.

This is a problem if the constructor enforces some condition. For example, a singleton object may be implemented so that the constructor can only be called once. As another example, database entities can be constructed so that they always come from a pool of managed instances.

You shouldn't implement your own mechanism for singletons. If you need a singleton, make an enumerated type with one instance which is, by convention, called `INSTANCE`.

```
public enum PersonDatabase {
    INSTANCE;

    public Person findById(int id) { . . . }
    . . .
}
```

This works because `enum` instances are guaranteed to be deserialized properly.

Now let's suppose that you are in the rare situation where you want to control the identity of each deserialized instance. As an example, suppose a `Person` class wants to restore its instances from a database when deserializing. Then you should not serialize the object itself. Instead, request that a proxy instance is saved. When restored, that proxy locates and constructs the desired object. Your class needs to provide a `writeReplace` method that returns the proxy object:

```
public class Person implements Serializable {
    private int id;
    // Other fields
    . . .
    @Serial private Object writeReplace() {
        return new PersonProxy(id);
    }
}
```

When a `Person` object is serialized, none of its fields are saved. Instead, the `writeReplace` method is called and *its return value* is serialized and written to the stream.

The proxy class needs to implement a `readResolve` method that yields a `Person` instance:

```
class PersonProxy implements Serializable {
    private int id;

    public PersonProxy(int id) {
         this.id = id;
    }
```

```
    @Serial private Object readResolve() {
        return PersonDatabase.INSTANCE.findById(id);
    }
}
```

When the `readObject` method finds a `PersonProxy` in an `ObjectInputStream`, it deserializes the proxy, calls its `readResolve` method, and returns the result.



Note: Unlike the `readObject` and `writeObject` methods, the `readResolve` and `writeReplace` methods need not be private.



Note: With enumerations and records, `readObject`/`writeObject` or `readExternal`/`writeExternal` methods are not used for serialization. With records, but not with enumerations, the `writeReplace` method will be used.

2.6.7. Versioning

If you use serialization to save objects, you need to consider what happens when your program evolves. Can version 1.1 read the old files? Can the users who still use 1.0 read the files that the new version is producing? Clearly, it would be desirable if object files could cope with the evolution of classes.

At first glance, it seems that this would not be possible. When a class definition changes in any way, its SHA fingerprint also changes, and you know that object input streams will refuse to read in objects with different fingerprints. However, a class can indicate that it is *compatible* with an earlier version of itself. To do this, you must first obtain the fingerprint of the *earlier* version of the class. Use the standalone `serialver` program that is part of the JDK to obtain this number. For example, running

```
serialver serial.Employee
```

prints

```
serial.Employee: static final long serialVersionUID = 8367346051156850807L;
```

All *later* versions of the class must define the `serialVersionUID` constant to the same fingerprint as the original.

```
class Employee implements Serializable { // version 1.1
    . . .
    @Serial public static final long serialVersionUID = 8367346051156850807L;
}
```

When a class has a static data member named `serialVersionUID`, it will not compute the fingerprint manually but will use that value instead.

Once that static data member has been placed inside a class, the serialization system is now willing to read in different versions of objects of that class.

If only the methods of the class change, there is no problem with reading the new object data. However, if the instance fields change, you may have problems. For example, the old file object may have more or fewer instance fields than the one in the program, or the types of the instance fields may be different. In that case, the object input stream makes an effort to convert the serialized object to the current version of the class.

The object input stream compares the instance fields of the current version of the class with those of the version in the serialized object. Of course, the object input stream considers only the nontransient and nonstatic instance fields. If two fields have matching names but different types, the object input stream makes no effort to convert one type to the other—the objects are incompatible. If the serialized object has instance fields that are not present in the current version, the object input stream ignores the additional data. If the current version has instance fields that are not present in the serialized object, the added fields are set to their default (`null` for objects, zero for numbers, and `false` for `boolean` values).

Here is an example. Suppose we have saved a number of employee records on disk, using the original version (1.0) of the class. Now we change the `Employee` class to version 2.0 by adding a instance field called `department`. **#ch01fig09** shows what happens when a 1.0 object is read into a program that uses 2.0 objects. The `department` field is set to `null`. **#ch01fig10** shows the opposite scenario: A program using 1.0 objects reads a 2.0 object. The additional `department` field is ignored.

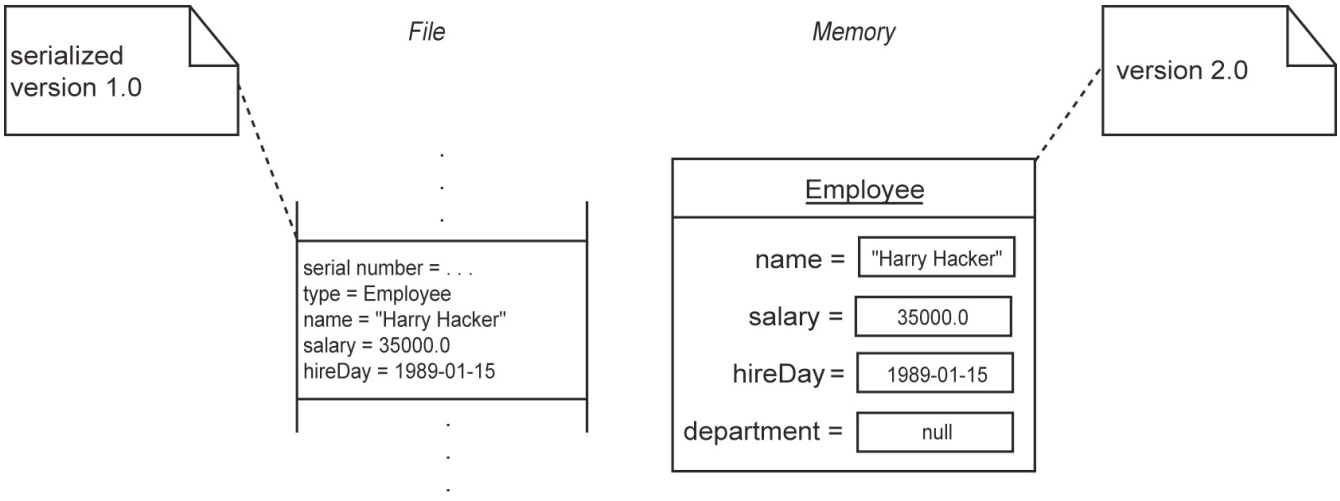


Figure 2.8: Reading an object with fewer instance fields

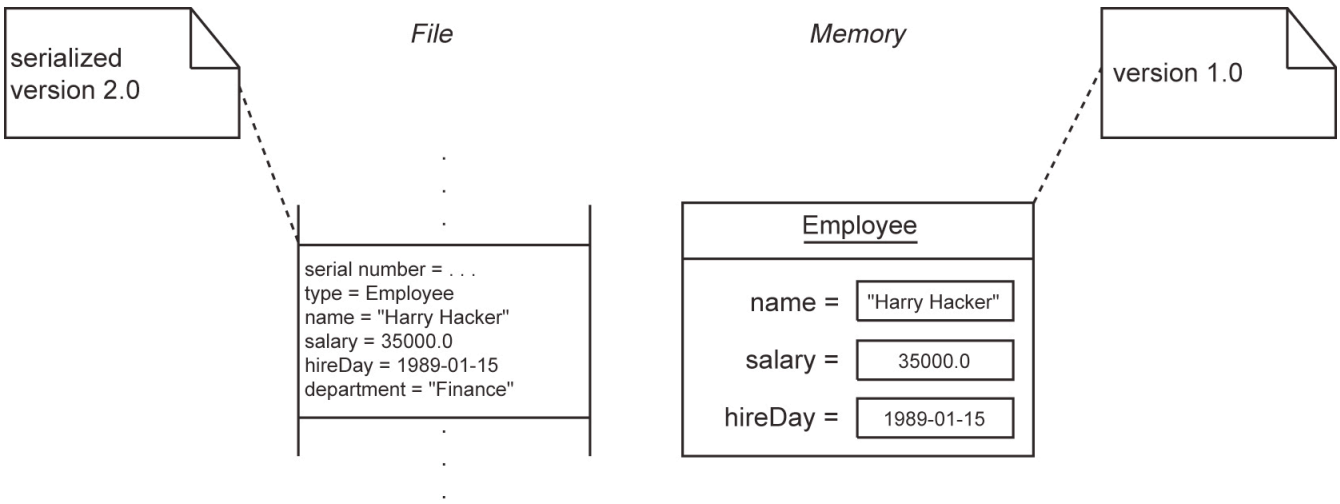


Figure 2.9: Reading an object with more instance fields

Is this process safe? It depends. Dropping a instance field seems harmless—the recipient still has all the data that it knows how to manipulate. Setting a instance field to `null` might not be so safe. Many classes work hard to initialize all instance fields in all constructors to non-`null` values, so that the methods don't have to be pre-

pared to handle `null` data. It is up to the class designer to implement additional code in the `readObject` method to fix version incompatibilities or to make sure the methods are robust enough to handle `null` data.



Tip: Before you add a `serialVersionUID` field to a class, ask yourself why you made your class serializable. If serialization is used only for short-term persistence, such as distributed method calls in an application server, there is no need to worry about versioning and the `serialVersionUID`. The same applies if you extend a class that happens to be serializable, but you have no intent to ever persist its instances. If your IDE gives you pesky warnings, change the IDE preferences to turn them off, or add an annotation `@SuppressWarnings("serial")`. This is safer than adding a `serialVersionUID` that you may later forget to change.



Note: Enumerations and records ignore the `serialVersionUID` field. An enumeration always has a `serialVersionUID` of `0L`. You can declare the `serialVersionUID` of a record, but the IDs don't have to match for deserialization.



Note: In this section, you saw what happens when the reader's version of a class has instance fields that aren't present in the object stream. It is also possible during class evolution for a superclass to be added. Then a reader using the new version may read an object stream in which the instance fields of the superclass are not set. By default, those instance fields are set to their zero/`null` default. That may leave the superclass in an unsafe state. The superclass can defend against that problem by defining an initialization method

```
@Serial private void readObjectNoData() throws ObjectStreamException
```

The method should either set the same state as the no-argument constructor or throw an `InvalidObjectException`. It is only called in the unusual circumstance that an object stream is read which contains an instance of a subclass with missing superclass data.

2.6.8. Deserialization and Security

During deserialization of a serializable class, objects are created without invoking any constructor of the class. Even if the class has a no-argument constructor, it is not used. The field values are set directly from the values of the object input stream.



Note: For serializable *records*, deserialization calls the canonical constructor, passing it the values of the components from the object input stream. (As a consequence, cyclic references in records are not restored.)

Bypassing constructors is a security risk. An attacker can craft bytes describing an invalid object that could have never been constructed. Suppose, for example, that the `Employee` constructor throws an exception when called with a negative salary. We would like to think that this means that no `Employee` object can have a negative salary. But, as you saw in the preceding section, it is not difficult to inspect the bytes for a serialized object and modify some of them. By doing so, one can craft bytes for an employee with a negative salary and then deserialize them.

A serializable class can optionally implement the `ObjectInputValidation` interface and define a `validateObject` method to check whether its objects are properly deserialized. For example, the `Employee` class can check that salaries are not negative:

```
public void validateObject() throws InvalidObjectException {
    IO.println("validateObject");
    if (salary < 0)
        throw new InvalidObjectException("salary < 0");
}
```

Unfortunately, the method is not invoked automatically. To invoke it, you must also provide the following method:

```
@Serial private void readObject(ObjectInputStream in)
        throws IOException, ClassNotFoundException {
    in.registerValidation(this, 0);
    in.defaultReadObject();
}
```

The object is then scheduled for validation, and the `validateObject` method is called when this object and all dependent objects have been loaded. The second parameter lets you specify a priority. Validation requests with higher priorities are done first.

There are other security risks. Adversaries can create data structures that consume enough resources to crash a virtual machine. More insidiously, any class on the class path can be deserialized. Hackers have been devious about piecing together "gadget chains", sequences of operations in various utility classes that use reflection and culminate in calling methods such as `Runtime.exec` with a string of their choice.

Any application that receives serialized data from untrusted sources over a network connection is vulnerable to such attacks. For example, some servers serialize session data and deserialize whatever data is returned in the HTTP session cookie.

You should avoid situations in which arbitrary data from untrusted sources are deserialized. In the example of session data, the server should sign the data, and only deserialize data with a valid signature.

JEP 290 and 415 provide a *serialization filter* mechanism to harden applications from such attacks. The filters can see the names of deserialized classes and several metrics (stream size, array sizes, total number of references, longest chain of references). Based on those data, the deserialization can be aborted.

In its simplest form, you provide a pattern describing the valid and invalid classes. For example, if you start our sample serialization demo as

```
java -Djdk.serialFilter='serial.*;java.**;!*' v2ch02.serial.ObjectStreamDemo
```

then the objects will be loaded. The filter allows all classes in the `serial` package and all classes whose package name starts with `java`, but no others. If you don't allow `java.**`, or at least `java.util.Date`, deserialization fails.

You can place the filter pattern into a configuration file and specify multiple filters for different purposes. You can also implement your own filters. See **https://docs.oracle.com/en/java/javase/21/core/serialization-filtering1.html** for details.

2.6.9. Using Serialization for Cloning

There is an amusing use for the serialization mechanism: It gives you an easy way to clone an object, provided the class is serializable. Simply serialize it to an output stream and then read it back in. The result is a new object that is a deep copy of the existing object. You don't have to write the object to a file—you can use a `ByteArrayOutputStream` to save the data into a byte array.

As **Listing 2.6** shows, to get `clone` for free, simply extend the `SerialCloneable` class, and you are done.

You should be aware that this method, although clever, will usually be much slower than a clone method that explicitly constructs a new object and copies or clones the instance fields.

Listing 2.6 v2ch02/serialClone/SerialCloneDemo.java

```
1  package v2ch02.serialClone;
2
3  class SerialCloneDemo {
4      void main() throws Exception {
5          var harry = new Employee("Harry Hacker", 35000, 1989, 10, 1);
6          // clone harry
7          var harry2 = (Employee) harry.clone();
8
9          // mutate harry
10         harry.raiseSalary(10);
11
12         // now harry and the clone are different
13         IO.println(harry);
14         IO.println(harry2);
15     }
16 }
```



Caution: Serialization attempts to restore cyclic references between objects, but this can lead to problems. Consider the following situation:

```
var a = new ArrayList<Object>();
var s = new HashSet<Object>();
a.add(s);
s.add(a); // Cyclic references between a and s
IO.println(SerialCloneable.serialClone(s).size()); // 1
IO.println(SerialCloneable.serialClone(a).size()); // NullPointerException
```

For convenience, this example uses the `serialClone` method from the sample code that serializes an object, and then deserializes it again.

Why does the second deserialization go awry? Array lists and hash sets do not serialize their internal data structure. Instead, their `writeObject` methods write out the serialized elements, and their `readObject` methods insert the deserialized elements into a new array or hash set. When `a` is deserialized in the last line, the following series of unfortunate events occurs:

1. The `size` field of the array list is restored from the stream, but `elements` is still `null`
2. A temporary array is allocated that later will be assigned to the `elements` field of the array list

3. Deserializing of the array elements commences. There is just one, the hash set
4. The hash set deserializes its sole element, the array list, obtaining a reference to the partially deserialized object
5. The hash set attempts to compute the hash code of the array list
6. The `ArrayList.hashCode` method throws a `NullPointerException` since the `elements` field is `null`

As you can see, the deserialization strategy for `ArrayList` is fundamentally flawed. It should not be possible to observe a partially constructed instance.

The deserialization strategy for `HashSet` has a different flaw, calling `hashCode` on elements that may only be partially deserialized.

In hindsight, serialization in Java has not been a success. It is complex and error-prone, and provides a significant attack surface. It is good to understand the mechanism, and its limitations. But you should think twice about using it for anything other than short-term persistence or secured communication between virtual machines of the same type.

2.7. Regular Expressions

Regular expressions are used to specify string patterns. You can use regular expressions whenever you need to locate strings that match a particular pattern. For example, one of our sample programs locates all hyperlinks in an HTML file by looking for strings of the pattern `<a href=". . ."></a>`.

Of course, when specifying a pattern, the `. . .` notation is not precise enough. You need to specify exactly what sequence of characters is a legal match, using a special syntax to describe a pattern.

In the following sections, we cover the regular expression syntax used by the Java API and discuss how to put regular expressions to work.

2.7.1. The Regular Expression Syntax

Let us start with a simple example. The regular expression

```
[Jj]ava.+
```

matches any string of the following form:

- The first letter is a `J` or `j`.
- The next three letters are `ava`.
- The remainder of the string consists of one or more arbitrary characters.

For example, the string `"javanese"` matches this particular regular expression, but the string `"Core Java"` does not.

As you can see, you need to know a bit of syntax to understand the meaning of a regular expression. Fortunately, for most purposes, a few straightforward constructs are sufficient.

In a regular expression, a character denotes itself unless it is one of the reserved characters

```
. * + ? { | ( ) [ \ ^ $
```

For example, the regular expression `Java` only matches the string `Java`.

The symbol `.` matches any single character. For example, `.a.a` matches `Java` and `data`.

The `*` symbol indicates that the preceding constructs may be repeated 0 or more times; for a `+`, it is 1 or more times. A suffix of `?` indicates that a construct is optional (0 or 1 times). For example, `be+s?` matches `be`, `bee`, and `bees`. You can specify other multiplicities with `{ }`—see **Table 2.13**.

A `|` denotes an alternative: `.(oo|ee)f` matches `beef` or `woof`. Note the parentheses—without them, `.oo|eef` would be the alternative between `.oo` and `eef`. Parentheses are also used for grouping—see **Section 2.7.4**.

A *character class* is a set of character alternatives enclosed in brackets, such as `[Jj]`, `[0-9]`, `[A-Za-z]`, or `[^0-9]`. Inside a character class, the `-` denotes a range (all characters whose Unicode values fall between the two bounds). However, a `-` that is the first or last character in a character class denotes itself. A `^` as the first character in a character class denotes the complement (all characters except those specified).

There are many *predefined character classes* such as `\d` (digits) or `\s` (spaces)— see **Table 2.13**. More complex character classes can be described with the `\p` syntax in **Table 2.14**. For example, `\p{Sc}` is the class of Unicode currency symbols.

The characters `^` and `$` match the beginning and end of input.

If you need to have a literal `. * + ? { | ( ) [ \ ^ $`, precede it by a backslash. Inside a character class, you only need to escape `[` and `\`, provided you are careful about the positions of `] - ^`. For example, `[]^-]` is a class containing all three of them.



Caution: If the regular expression is in a string literal, each backslash needs to be escaped with another backslash. If you forget that second backslash, you usually get an error because sequences such as `\$` or `\.` are not valid in string literals. But if you want to match a word boundary and accidentally use `\b` instead of `\\b`, then you have a problem: `\b` is a valid escape sequence, indicating a backspace.

Your IDE can help with that escaping. Paste the regular expression string into an empty string, and the backslashes will be doubled.

Instead of using backslashes, you can surround a string with `\Q` and `\E`. For example, `\(\$0\.99\)` and `\Q($0.99)\E` both match the string `($0.99)`.



Tip: If you have a string that may contain some of the many special characters in the regular expression syntax, you can escape them all by calling `Parse.quote(str)`. This simply surrounds the string with `\Q` and `\E`, but it takes care of the special case where `str` may contain `\E`.

Table 2.13: Regular Expression Syntax

Expression	Description	Example
Characters		

Expression	Description	Example
c, not one of `. * + ? { | ( ) [ \ ^ $`	The character c	`J`
`.`	Any character except line terminators, or any character if the `DOTALL` flag is set	
`\X`	Any Unicode "extended grapheme cluster", which is perceived as a character or symbol	
`\x{` p `}`	The Unicode code point with hex code p	`\x{1D546}`
`\u` hhhh, `\x` hh, `\0` o, `\0` oo, `\0` ooo	The UTF-16 code unit with the given hex or octal value	`\uFEFF`
`\a`, `\e`, `\f`, `\n`, `\r`, `\t`	Alert (`\x{7}`), escape (`\x{1B}`), form feed (`\x{B}`), newline (`\x{A}`), carriage return (`\x{D}`), tab (`\x{9}`)	`\n`
`\c` c, where c is in `[A-Z]` or one of `@ [ \ ] ^ _ ?`	The control character corresponding to the character c	`\cH` is a backspace (`\x{8}`)
`\` c, where c is not in `[A-Za-z0-9]`	The character c	`\\`
`\Q. . .\E`	Everything between the start and the end of the quotation	`\Q(. . .)\E` matches the string `(. . .)`
Character Classes		
`[` C_1C_2 `. . .]`, where C_i are characters, ranges c - d, or character classes	Any of the characters represented by C_1, C_2, . . .	`[0-9+-]`
`[^. . .]`	Complement of a character class	`[^\d\s]`
`[. . .&&. . .]`	Intersection of character classes	`[\p{L}&&[^A-Za-z]]`
`\p{. . .}`, `\P{. . .}`	A predefined character class (see Table 2.14); its complement	`\p{L}` matches a Unicode letter, and so does `\pL`—you can omit braces around a single letter

Expression	Description	Example
`\d`, `\D`	Digits (`[0-9]`, or `\p{Digit}` when the `UNICODE_CHARACTER_CLASS` flag is set); the complement	`\d+` is a sequence of digits
`\w`, `\W`	Word characters (`[a-zA-Z0-9_]`, or Unicode word characters when the `UNICODE_CHARACTER_CLASS` flag is set); the complement	
`\s`, `\S`	Spaces (`[ \n\r\t\f\x{B}]`, or `\p{IsWhite_Space}` when the `UNICODE_CHARACTER_CLASS` flag is set); the complement	`\s*,\s*` is a comma surrounded by optional white space
`\h`, `\v`, `\H`, `\V`	Horizontal whitespace, vertical whitespace, their complements	
Sequences and Alternatives		
XY	Any string from `X`, followed by any string from `Y`	`[1-9][0-9]*` is a positive number without leading zero
X | Y	Any string from `X` or `Y`	`http|ftp`
Grouping		
(X)	Captures the match of X	`'([^']*)'` captures the quoted text
\ n	The nth group	`(['"]).*\1` matches `'Fred'` or `"Fred"` but not `"Fred'`
(?< name > X)	Captures the match of X with the given name	`'(?<id>[A-Za-z0-9]+)'` captures the match with name `id`
`\k<` name `>`	The group with the given name	`\k<id>` matches the group with name `id`
(?: X)	Use parentheses without capturing X	In `(?:http|ftp)://(.*)`, the match after `://` is `\1`
(? f_1f_2 . . .: X), (? f_1 . . .- f_k . . .: X), with f_i in `[dimsuUx]`	Matches, but does not capture, X with the given flags on or off (after `-`)	`(?i:jpe?g)` is a case-insensitive match

Expression	Description	Example
Other `(?. . .)`	See the `Pattern` API documentation	
Quantifiers		
X ?	Optional X	`\+?` is an optional + sign
X `*`, X `+`	0 or more X, 1 or more X	`[1-9][0-9]+` is an integer ≥ 10
X { n }, X { n ,}, X { m , n }	n times X, at least n times `X`, between m and n times X	`[0-7]{1,3}` are one to three octal digits
Q `?`, where Q is a quantified expression	Reluctant quantifier, attempting the shortest match before trying longer matches	`.*(<.+?>).*` captures the shortest sequence enclosed in angle brackets
Q +, where Q is a quantified expression	Possessive quantifier, taking the longest match without backtracking	`'[^']*+'` matches strings enclosed in single quotes and fails quickly on strings without a closing quote
Boundary Matches		
`^ $`	Beginning, end of input (or beginning, end of line in multiline mode)	`^Java$` matches the input or line `Java`
`\A \z \Z`	Beginning of input, end of input, end of input before final line terminator	
`\b \B`	Word boundary, nonword boundary	`\bJava\b` matches the word `Java`
`\b{g}`	Grapheme cluster boundary	Useful with `split` to decompose a string into grapheme clusters
`\R`	A Unicode line break	
`\G`	The end of the previous match	

Table 2.14: Predefined Character Classes `\p{. . .}`

Name	Description
posixClass	posixClass is one of `Lower`, `Upper`, `Alpha`, `Digit`, `Alnum`, `Punct`, `Graph`, `Print`, `Cntrl`, `XDigit`, `Space`, `Blank`, `ASCII`, interpreted as POSIX or Unicode class, depending on the `UNICODE_CHARACTER_CLASS` flag

Name	Description
`Is` Script, `sc=` Script, `script=` Script	A script accepted by `Character.UnicodeScript.forName`
`In` Block, `blk=` Block, `block=` Block	A block accepted by `Character.UnicodeBlock.forName`
Category, `In` Category, `gc=` Category, `general_category=` Category	A one- or two-letter name for a Unicode general category
`Is` Property	Property is one of `Alphabetic`, `Ideographic`, `Letter`, `Lowercase`, `Uppercase`, `Titlecase`, `Punctuation`, `Control`, `White_Space`, `Digit`, `Hex_Digit`, `Join_Control`, `Noncharacter_Code_Point`, `Assigned`
`java` Method	Invokes the method `Character.is` Method (must not be deprecated)

Unfortunately, the regular expression syntax is not completely standardized between various programs and libraries; there is a consensus on the basic constructs but many maddening differences in the details. The Java regular expression classes use a syntax that is similar to, but not quite the same as, the one used in the Perl language. **Table 2.13** shows all constructs of the Java syntax. For more information on the regular expression syntax, consult the API documentation for the `Pattern` class or the book Mastering Regular Expressions by Jeffrey E. F. Friedl (O'Reilly and Associates, 2006).

2.7.2. Testing a Match

Generally, there are two ways to use a regular expression: Either you want to find out whether a string conforms to the expression, or you want to find all matches of a regular expressions in a string.

In the first case, simply use the static `matches` method:

```
String regex = "-?\\d+";
CharSequence input = . . .;
if (Pattern.matches(regex, input)) {
    . . .
}
```

If you need to use the same regular expression many times, it is more efficient to compile it. Then, create a `Matcher` for each input:

```
Pattern pattern = Pattern.compile(regex);
Matcher matcher = pattern.matcher(input);
if (matcher.matches()) . . .
```

If the match succeeds, you can retrieve the location of matched groups—see the following section.

If you want to test whether the input *contains* a match, use the `find` method instead:

```
if (matcher.find()) . . .
```

The `find` and `match` methods mutate the state of the `Matcher` object. If you just want to find out whether a given `Matcher` has already found a match, call the `hasMatch` method instead.

You can turn the pattern into a predicate:

```
Pattern digits = Pattern.compile("[0-9]+");
List<String> strings = List.of("December", "31st", "1999");
List<String> matchingStrings
    = strings.stream().filter(digits.asMatchPredicate()).toList(); // ["1999"]
```

The result contains all strings that match the regular expression.

Use the `asPredicate` method to test whether a string contains a match:

```
List<String> stringsContainingMatch
    = strings.stream().filter(digits.asPredicate()).toList(); // ["31st", "1999"]
```

2.7.3. Finding All Matches in a String

In this section, we consider the other common use case for regular expressions—finding all matches in an input. Use this loop:

```
String input = . . .;
Matcher matcher = pattern.matcher(input);
while (matcher.find()) {
    String match = matcher.group();
    int matchStart = matcher.start();
    int matchEnd = matcher.end();
    . . .
}
```

In this way, you can process each match in turn. As shown in the code fragment, you can get the matched string as well as its position in the input string.

More elegantly, you can call the `results` method to get a `Stream<MatchResult>`. The `MatchResult` interface has methods `group`, `start`, and `end`, just like `Matcher`. (In fact, the `Matcher` class implements this interface.) Here is how you get a list of all matches:

```
List<String> matches = pattern.matcher(input)
    .results()
    .map(MatchResult::group)
    .toList();
```

If you have the data in a file, then you can use the `Scanner.findAll` method to get a `Stream<MatchResult>`, without first having to read the contents into a string. You can pass a `Pattern` or a pattern string:

```
Scanner in = new Scanner(path, "UTF_8");
Stream<String> words = in.findAll("\\pL+")
    .map(MatchResult::group);
```

Listing 2.7 puts this mechanism to work. It locates all hypertext references in a web page and prints them. To run the program, supply a URL on the command line, such as

```
java v2ch02.regex.HrefMatch https://horstmann.com/index.html
```

Listing 2.7 v2ch02/regex/HrefMatch.java

```
1  package v2ch02.regex;
2
3  import module java.base;
4
5  /**
6   * This program displays all hyperlinks in a web page by matching a regular expression that
7   * describes the <a href=...> HTML tag.
8   */
9  class HrefMatch {
10     void main(String[] args) throws Exception {
11         String urlString = args.length > 0 ? args[0] : "https://openjdk.org/";
12
13         // read contents of URL
14         InputStream in = URI.create(urlString).toURL().openStream();
15         var input = new String(in.readAllBytes());
16
17         // search for all occurrences of pattern
18         String patternString = "<a\\s+href\\s*=\\s*(\"[^\"]*\"|[^\\s>]*)\\s*>";
19         Pattern pattern = Pattern.compile(patternString, Pattern.CASE_INSENSITIVE);
20         pattern.matcher(input).results().map(MatchResult::group).forEach(IO::println);
21     }
22 }
```

2.7.4. Groups

It is common to use groups for extracting components of a match. For example, suppose you have a line item in the invoice with item name, quantity, and unit price such as

```
Blackwell Toaster    USD29.95
```

Here is a regular expression with groups for each component:

```
(\p{Alnum}+(\s+\p{Alnum}+)*)\s+([A-Z]{3})([0-9.]*)
```

After matching, you can extract the `n` th group from the matcher as

```
String contents = matcher.group(n);
```

Groups are ordered by their opening parenthesis, starting at 1. (Group 0 is the entire input.) In this example, here is how to take the input apart:

```
Matcher matcher = pattern.matcher(input);
if (matcher.matches()) {
    item = matcher.group(1);
    currency = matcher.group(3);
    price = matcher.group(4);
}
```

We aren't interested in group 2; it only arose from the parentheses that were required for the repetition. For greater clarity, you can use a noncapturing group:

```
(\p{Alnum}+(?:\s+\p{Alnum}+)*)\s+([A-Z]{3})([0-9.]*)
```

Or, even better, use named groups:

```
(?<item>\p{Alnum}+(\s+\p{Alnum}+)*)\s+(?<currency>[A-Z]{3})(?<price>[0-9.]*)
```

Then, you can retrieve the items by name:

```
item = matcher.group("item");
```

With the `start` and `end` methods, you can get the group positions in the input:

```
int itemStart = matcher.start("item");
int itemEnd = matcher.end("item");
```

As of Java 20, the `MatchResult` class also supports named groups.

The `namedGroups` method yields a `Map<String, Integer>` from group names to numbers.

Note: When you have a group inside a repetition, such as `(\s+\p{Alnum}+)*` in the example above, it is not possible to get all of its matches. The `group` method only yields the last match, which is rarely useful. You need to capture the entire expression with another group.

Tip: You can add comments to regular expressions. A comment starts with `#` and extends until the end of the line. This works particularly well with text blocks:

```
var regex = """
([1-9]|1[0-2]) #hours
:([0-5][0-9]) #minutes
[ap]m""";
```

Listing 2.8 prompts for a pattern, then for strings to match. It prints out whether or not the input matches the pattern. If the input matches and the pattern contains groups, the program prints the group boundaries as

parentheses, for example:

```
((11):(59))am
```

Listing 2.8 v2ch02/regex/RegexDemo.java

```
package v2ch02.regex;

/**
 * This program tests regular expression matching. Enter a pattern and strings to match,
 * or hit Enter to exit. If the pattern contains groups, the group boundaries are displayed
 * in the match.
 */

import module java.base;

class RegexDemo {
    void main() throws Exception {
        String patternString = IO.readln("Enter pattern: ");

        Pattern pattern = Pattern.compile(patternString);

        while (true) {
            String input = IO.readln("Enter string to match: ");
            if (input == null || input.equals("")) return;
            Matcher matcher = pattern.matcher(input);
            if (matcher.matches()) {
                IO.println("Match");
                int g = matcher.groupCount();
                if (g > 0) {
                    for (int i = 0; i < input.length(); i++) {
                        // Print any empty groups
                        for (int j = 1; j <= g; j++)
                            if (i == matcher.start(j) && i == matcher.end(j)) IO.print("()");
                        // Print ( for non-empty groups starting here
                        for (int j = 1; j <= g; j++)
                            if (i == matcher.start(j) && i != matcher.end(j)) IO.print("(");
                        IO.print(input.charAt(i));
                        // Print ) for non-empty groups ending here
                        for (int j = 1; j <= g; j++)
                            if (i + 1 != matcher.start(j) && i + 1 == matcher.end(j))
                                IO.print(")");
                    }
                    IO.println();
                }
            }
            else
                IO.println("No match");
        }
    }
}
```

2.7.5. Splitting along Delimiters

Sometimes, you want to break an input along matched delimiters and keep everything else. The `Pattern.split` method automates this task. You obtain an array of strings, with the delimiters removed:

```
String input = "1, 2 , 3,4";
Pattern commas = Pattern.compile("\\s*,\\s*");
String[] tokens = commas.split(input); // ["1", "2", "3", "4"]
```

If there are many tokens, you can fetch them lazily:

```
Stream<String> tokens = commas.splitAsStream(input);
```

To also capture the delimiters, use the `splitWithDelimiters` method:

```
tokens = commas.splitWithDelimiters(input, -1); // ["1", ", ", "2", " , ", "3", ",", "4"]
```

If the second argument is a positive number `n`, the separator pattern is applied at most `n` - 1 times, and the last element is the remaining string. Otherwise, the pattern is applied as many times as possible. With a limit of zero, trailing empty strings are discarded.

If you don't care about precompiling the pattern or lazy fetching, you can just use the `split` and `splitWithDelimiter` methods of the `String` class:

```
tokens = input.split("\\s*,\\s*");
```



Caution: It is easy to forget that the argument of `split` is a regular expression. For example,

```
"com.horstmann.corejava".split(".")
```

does not split along the dots. Instead, every character is a separator, and the result is an empty array!

You need to escape the dot with a backslash in the regular expression, and therefore with two backslashes in the string literal:

```
"com.horstmann.corejava".split("\\.")
```

Alternatively, use the `Pattern.quote` method:

```
"com.horstmann.corejava".split(Pattern.quote("."));
```

If the input is in a file, use a scanner:

```
Scanner in = new Scanner(path, "UTF_8");
in.useDelimiter("\\s*,\\s*");
Stream<String> tokens = in.tokens();
```

2.7.6. Replacing Matches

If you want to replace all matches of a regular expression with a string, call `replaceAll` on the matcher:

```
Matcher matcher = commas.matcher(input);
String result = matcher.replaceAll(",");
    // Normalizes the commas
```

Or, if you don't care about precompiling, use the `replaceAll` method of the `String` class.

```
String result = input.replaceAll("\\s*,\\s*", ",");
```

The replacement string can contain group numbers `$n` or names `${name}`. They are replaced with the contents of the corresponding captured groups.

```
String result = "3:45".replaceAll(
    "(\\d{1,2}):(?<minutes>\\d{2})",
    "$1 hours and ${minutes} minutes");
    // Sets result to "3 hours and 45 minutes"
```

You can use `\` to escape `$` and `\` in the replacement string, or you can call the `Matcher.quoteReplacement` convenience method:

```
matcher.replaceAll(Matcher.quoteReplacement(str))
```

If you want to carry out a more complex operation than splicing in group matches, then you can provide a replacement function instead of a replacement string. The function accepts a `MatchResult` and yields a string. For example, here we replace all words with at least four letters with their uppercase version:

```
String result = Pattern.compile("\\pL{4,}")
    .matcher("Mary had a little lamb")
    .replaceAll(m -> m.group().toUpperCase());
    // Yields "MARY had a LITTLE LAMB"
```

The `replaceFirst` method replaces only the first occurrence of the pattern.

2.7.7. Flags

Several *flags* change the behavior of regular expressions. You can specify them when you compile the pattern:

```
Pattern pattern = Pattern.compile(regex,
    Pattern.CASE_INSENSITIVE | Pattern.UNICODE_CHARACTER_CLASS);
```

Or you can specify them inside the pattern:

```
String regex = "(?iU:expression)";
```

Here are the flags:

- `Pattern.CASE_INSENSITIVE` or `i`: Match characters independently of the letter case. By default, this flag takes only US ASCII characters into account.
- `Pattern.UNICODE_CASE` or `u`: When used in combination with `CASE_INSENSITIVE`, use Unicode letter case for matching.
- `Pattern.UNICODE_CHARACTER_CLASS` or `U`: Select Unicode character classes instead of POSIX. Implies `UNICODE_CASE`.
- `Pattern.MULTILINE` or `m`: Make `^` and `$` match the beginning and end of a line, not the entire input.
- `Pattern.UNIX_LINES` or `d`: Only `'\n'` is a line terminator when matching `^` and `$` in multiline mode.
- `Pattern.DOTALL` or `s`: Make the `.` symbol match all characters, including line terminators.
- `Pattern.COMMENTS` or `x`: Whitespace and comments (from `#` to the end of a line) are ignored.
- `Pattern.LITERAL`: The pattern is taken literally and must be matched exactly, except possibly for letter case.
- `Pattern.CANON_EQ`: Take canonical equivalence of Unicode characters into account. For example, u followed by ¨ (diaeresis) matches ü.

The last two flags cannot be specified inside a regular expression.

`java.util.regex.Pattern` 1.4

- `static Pattern compile(String expression)`
- `static Pattern compile(String expression, int flags)`
  compile the regular expression string into a pattern object for fast processing of matches. The `flags` parameter has one or more of the bits `CASE_INSENSITIVE`, `UNICODE_CASE`, `MULTILINE`, `UNIX_LINES`, `DOTALL`, and `CANON_EQ` set.
- `Matcher matcher(CharSequence input)`
  returns a `matcher` object that you can use to locate the matches of the pattern in the input.
- `String[] split(CharSequence input)`
- `String[] split(CharSequence input, int limit)`
- `String[] splitWithDelimiters(CharSequence input, int limit)`
- `Stream<String> splitAsStream(CharSequence input)` 8
  split the input string into tokens, with the pattern specifying the form of the delimiters. Return an array or stream of tokens. The delimiters are not part of the tokens. The second form has a parameter `limit` denoting the maximum number of strings to produce. If `limit - 1` matching delimiters have been found, then the last entry of the returned array contains the remaining unsplit input. If `limit` is $\leq$ `0`, then the entire input is split. If `limit` is `0`, then trailing empty strings are not placed in the returned array.
- `Map<String, Integer> namedGroups()` 20
  yields an immutable map from group names to group indexes.

`java.util.regex.Matcher` 1.4

- `boolean matches()`
  returns `true` if the input matches the pattern.
- `boolean lookingAt()`
  returns `true` if the beginning of the input matches the pattern.
- `boolean find()`
- `boolean find(int start)`
  attempt to find the next match and return `true` if another match is found.
- `boolean hasMatch()` 20
  `true` if this matcher is positioned on a match as a result of calling `matches` or `find`.
- `String replaceAll(String replacement)`
- `String replaceFirst(String replacement)`
  return a string obtained from the matcher input by replacing all matches, or the first match, with the replacement string. The replacement string can contain references to pattern groups as `$n`. Use `\$` to include a `$` symbol.
- `static String quoteReplacement(String str)` 5.0
  quotes all `\` and `$` in `str`.
- `String replaceAll(Function<MatchResult,String> replacer)` 9
  replaces every match with the result of the `replacer` function applied to the `MatchResult`.
- `Stream<MatchResult> results()` 9
  yields a stream of all match results.

java.util.regex.MatchResult 5.0

- `String group()`
- `String group(int group)`
- `String group(String name)` 20
  yield the matched string or the string matched by the given group.
- `int start()`
- `int end()`
- `int start(int group)`
- `int start(String nane)` 20
- `int end(int group)`
- `int end(String name)` 20
  yield the start and end offsets of the matched string or the string matched by the given group.
- `Map<String, Integer> namedGroups()` 20
  yields an immutable map from group names to group indexes of the underlying pattern.

`java.util.Scanner` 5.0

- `Stream<MatchResult> findAll(Pattern pattern)` 9
  yields a stream of all matches of the given pattern in the input produced by this scanner.

You have now seen how to carry out input and output operations in Java, and had an overview of the regular expression package that was a part of the “new I/O” specification. In the next chapter, we turn to the processing of XML data.

Previous chapter
< 1. Streams

Next chapter
3. XML >